

CheckpointTensorImpl.h&cpp代码分析

CheckpointTensorImpl.h

```
1  #pragma once
2
3  /*标准库引用*/
4  #include <atomic> // 支持原子操作，用于实现线程安全。
5  #include <memory> // 提供智能指针（如 std::shared_ptr 和 std::unique_ptr）。
6  #include <numeric> // 提供数值运算的标准库算法（如 std::accumulate）。
7  #include <random> // 提供随机数生成功能。
8
9  /*c10 库引用*/
10 #include <c10/core/Backend.h> // 定义了不同的计算后端（如 CPU、CUDA）。
11 #include <c10/core/CopyBytes.h> // 提供字节拷贝功能。
12 #include <c10/core/DispatchKeySet.h> // 用于分派机制（dispatch mechanism）。
13 #include <c10/core/MemoryFormat.h> // 定义了内存格式（如 contiguous）。
14 #include <c10/core/Storage.h> // 管理张量存储的底层抽象。
15 #include <c10/core/TensorImpl.h> // 张量实现的基础定义。
16 #include <c10/core/TensorOptions.h> // 提供创建张量时的选项（如数据类型、设备）。
17 #include <c10/core/impl/LocalDispatchKeySet.h>
18 #include <c10/util/Exception.h> // 异常处理工具。
19 #include <c10/util/Flags.h> // 处理标志位。
20 #include <c10/util/Logging.h> // 日志记录功能。
21 #include <c10/util/Optional.h> // 实现 optional 类型，用于可能不存在的值。
22 #include <c10/util/python_stub.h> // 用于在不依赖 Python 解释器的情况下定义一些
    Python 相关的函数。
23
24 /*ATen 库引用*/
25 #include <ATen/ATen.h>
26 #include <ATen/Tensor.h>
27
28 /*宏定义*/
29 // likely(x) 和 unlikely(x) 宏定义是用于分支预测优化的 GCC
30 // 编译器扩展。它们提示编译器某个条件更可能为真或假，从而优化生成的机器代码。
31 #define likely(x) __builtin_expect(!!(x), 1)
32 #define unlikely(x) __builtin_expect(!!(x), 0)
33 // TORCH_CHECK 宏在这段代码中被定义为空宏。
34 // 通常，这个宏用于在调试和开发过程中进行条件检查，并在条件不满足时抛出错误。这里定义为空
    可能是因为文件处于某种剖析（profile）模式，不需要运行时检查。
35 // #define TORCH_CHECK(a, ...) // profile mode 配置文件模式
36
```

```
37 // System Description:
38 // Every Tensor is managed by a CheckpointTensor, that describe how it is
39 // computed, (the function and the inputs) and might optionally hold the
   tensor
40 // value. The tensor value might be dropped, and when requested later,
41 // recomputed and cached again.
42
43 // 系统描述:
44 // 每个 Tensor 都由 CheckpointTensor
45 // 管理, 它描述了它的计算方式 (函数和输入), 并且可以选择保存张量值。
46 // 张量值可能会被删除, 并在稍后请求时重新计算并再次缓存。
47
48 // Corner Cases:
49 // A CheckpointedTensor might require_grad.
50 //   In this case the underlying data must not require_grad, as we want
51 //   backpropagation on the outer, uncheckpoined level. To be more specific,
52 //   suppose a tensor is recomputed multiple times. We want to only compute
   the
53 //   gradient exactly once. To do this, the wrapper must be require_grad, and
54 //   the wrapped value must not.
55 // A CheckpointedTensor might be constant.
56 //   In this case it is unevictable.
57 // An operator might return multiple output.
58 //   In this case the computation info (rematerializer) is shared between all
   of
59 //   them, and when the function get computed again all value get cached.
60 // An operator might not return value, but only mutate input value.
61 //   To combat this, we COW the operator, and wrap CheckpopintTensor with a
   Ref.
62 //   By doing this the inner CheckpointTensor is kept purely functional.
63 // An operator might try to mutate uncheckpointed tensor.
64 //   We do not support this and will error.
65 // An operator might create aliases.
66 //   We track alias in AliasPool.
67 //   Each AliasPool hold a set of tensor that is alias to eachother.
68 // An operator might try to create Alias to an unevictable tensor.
69 //   In such a case the output tensor is unevictable.
70 // An operator might try to mutate Tensor with Alias.
71 //   We do not support this case an will error if a Tensor has any alive
   Alias.
72 //   However it could be done without a major redesign of the system.
73 //   Each AliasPool will hold weak pointers to the External Reference.
74 //   When alias mutation occur, we make a rematerialize_function that take in
75 //   the base tensor (other tensor alias from) and output all the new value of
76 //   the aliases, then update the Ref. Of course, the cleaner way is to not
77 //   support this. Shame on those who use this feature.
78
```

```
79 // 极端情况:
80 // CheckpointedTensor 可能 require_grad。
81 // 在这种情况下, 底层数据不能 require_grad, 因为我們希望在外部、uncheckpointed
82 // 级别上进行反向传播。
83 // 更具体地说, 假设一个张量被重新计算多次。我们只想计算一次梯度。
84 // 为此, wrapper 必须是 require_grad, 而包装的值则不能。
85 // CheckpointedTensor 可能是常量。
86 // 在这种情况下它是 unevictable 的。
87 // operator可能返回多个输出。
88 // 在这种情况下, 计算信息 (rematerializer) 在它们之间共享, 当函数再次计算时, 所有值
    都会被缓存。
89 // operator可能不会返回值, 而只会改变输入值。
90 // 为了解决这个问题, 我们 COW the operator, 并用 Ref 包装 CheckpointTensor。
91 // 通过这样做, 内部 CheckpointTensor 保持纯粹的功能。
92 // operator可能会尝试mutate uncheckpointed的张量。
93 // 我们不支持这一点并且会出错。
94 // operator可能会创建aliases。
95 // 我们在 AliasPool 中跟踪aliases。
96 // 每个 AliasPool 都保存一组彼此aliases的张量。
97 // operator可能会尝试为 unevictable 的张量创建 Alias。
98 // 在这种情况下, 输出张量是 unevictable 的。
99 // operator可能会尝试使用 Alias 来改变 Tensor。
100 // 我们不支持这种情况, 如果张量有任何活动的aliases, 则会出现错误。
101 // 然而, 无需对系统进行重大重新设计即可完成。
102 // 每个 AliasPool 将保存指向External引用的弱指针。
103 // 当发生别名突变时, 我们创建一个 rematerialize_function
104 // , 它接收基础张量 (来自其他张量别名) 并输出别名的所有新值, 然后更新 Ref。
105 // 当然, 更简洁的方法是不支持这一点。
106 // 对那些使用此功能的人感到羞耻。
107
108 // Memory Safety:
109 // The objects here will have lots of backedges.
110 // In order to collect memory when computation is completed, We require that
    all
111 // strong pointer is of the form of value -> input. This ensure that
    everything
112 // will be released if there is no external ref whatsoever.
113
114 // 内存安全:
115 // 这里的对象将有很多 backedges。
116 // 为了在计算完成时收集内存, 我们要求所有strong指针都是 value -> input
117 // 的形式。 这确保了如果没有任何外部引用, 所有内容都会被释放。
118
119 // Optimization:
120 // We treat tensor that has no external reference differently -
121 // They will never be externally used again so we assume their next use time
    is
```

```

122 // infinite so, if it doesnt has any evicted neighbor it will get evicted
123 // immediately.
124
125 // 优化:
126 // 我们以不同的方式对待没有外部参考的张量 -
127 // 它们永远不会再被外部使用, 所以我们假设它们的下一次使用时间是无限的
128 // 因此, 如果它没有任何被驱逐的邻居, 它将立即被驱逐。
129
130 // Note: to code fast I do not use RAII and just assume the code will not try
    to
131 // recover from exception. It should be easy to fix though.
132
133 // 注意: 为了快速编码, 我不使用 RAII, 只是假设代码不会尝试从异常中恢复。
134 // 不过修复起来应该很容易。
135
136 namespace at {
137
138 // TODO: using a pool allocator might make more sense - no need to allocate
    and
139 // delete each pointer individually.
140 // TODO: 使用池分配器可能更有意义 - 无需单独分配和删除每个指针。
141 /*模板结构体 EquivalentClassNode, 用于表示等价类节点。*/
142 template <typename T> // 一个模板结构体, T 是模板参数。
143 // EquivalentClassNode 可以作为 intrusive_ptr 的目标类型, 即将
144 // EquivalentClassNode 类型的对象存储在 intrusive_ptr 中。
145 struct EquivalentClassNode : intrusive_ptr_target {
146     // 构造函数, 接收一个类型为 T 的参数 t, 并用它初始化成员变量 t_unsafe。使用
147     // explicit 关键字表明此构造函数不允许隐式转换。
148     explicit EquivalentClassNode(const T& t)
149         : t_unsafe(t) {} // 显式 (explicit) 构造函数
150     // 成员变量 parent 是一个指向 EquivalentClassNode 类型的 intrusive_ptr, 用
151     // mutable 修饰, 表示即使在 const 成员函数中也可以修改 parent。
152     /*
153     这是一种 组合 关系, 因为 parent 是 EquivalentClassNode
154     的一部分, 其生命周期依赖于 EquivalentClassNode 对象的生命周期。
155     */
156     mutable intrusive_ptr<EquivalentClassNode> parent;
157     // 检查当前节点是否是根节点。如果 parent 为空 (即当前节点没有父节点), 则返回
158     // true, 否则返回 false。
159     bool is_root() {
160         return !parent;
161     }
162     // 一个重载的虚函数, 重置 parent 指针, 释放资源。
163     void release_resources() override {
164         parent.reset();
165     }
166     // t_unsafe 是类型为 T 的成员变量, 存储构造函数传入的值。

```

```

167     T t_unsafe;
168 };
169
170 /*模板函数，返回给定节点 n 的根节点的 t_unsafe 引用。*/
171 template <typename T>
172 T& get_t(const intrusive_ptr<EquivalentClassNode<T>>& n) {
173     return find_root(n)->t_unsafe;
174 }
175
176 /*模板函数，更新给定节点 n 的根节点的 t_unsafe 成员变量。*/
177 template <typename T>
178 static void update_t(
179     const intrusive_ptr<EquivalentClassNode<T>>& n,
180     const T& t) {
181     find_root(n)->t_unsafe = t;
182 }
183
184 /*模板函数，找到给定节点 n 的根节点。*/
185 template <typename T>
186 intrusive_ptr<EquivalentClassNode<T>> find_root(
187     const intrusive_ptr<EquivalentClassNode<T>>& n) {
188     if (n->is_root()) {
189         return n;
190     } else { // 如果当前节点不是根节点，则递归查找其父节点的根节点，并将当前节点的父节点
        // 更新为根节点。这是路径压缩的一种形式，用于加速未来的根节点查找。
191         n->parent = find_root(n->parent);
192         return n->parent;
193     }
194 }
195
196 /*模板函数，合并两个等价类节点，并返回合并后的根节点。*/
197 template <typename T>
198 intrusive_ptr<EquivalentClassNode<T>> merge(
199     const std::function<T(const T&, const T&)>&
200         merge_t, // merge_t: 一个函数对象，用于合并两个 T 类型的值。
201     const intrusive_ptr<EquivalentClassNode<T>>&
202         lhs, // 左操作数，指向 EquivalentClassNode<T> 类型节点的智能指针。
203     const intrusive_ptr<EquivalentClassNode<T>>&
204         rhs) { // 右操作数，指向 EquivalentClassNode<T> 类型节点的智能指针。
205     // 查找根节点
206     auto l = find_root(lhs);
207     auto r = find_root(rhs);
208     // 如果 lhs 和 rhs 的根节点相同，说明它们已经在同一个等价类中，直接返回根节点 l。
209     if (l == r) {
210         return l;
211     }
212     // 合并等价类，将 lhs 的根节点挂到 rhs 的根节点下面。

```

```

213     l->parent = r;
214     // 使用 merge_t 函数合并 l->t_unsafe 和 r->t_unsafe, 并将结果存储到
215     // r->t_unsafe 中。
216     r->t_unsafe = merge_t(l->t_unsafe, r->t_unsafe);
217     return r;
218 }
219
220 /*在C++中, 符号 &
221     表示引用 (reference)。在函数声明中, 使用引用作为参数有几个重要的意义:
222     避免复制:
223     传递对象的引用而不是对象本身, 避免了对象的复制操作, 提高了效率, 尤其当对象很大时, 复制
    开销会非常高。
224     允许修改:
225     引用可以用来在函数内部修改传递进来的对象。如果不想允许函数修改对象, 可以使用const引
    用。
226     统一性:
227     引用与原对象共享内存地址, 操作引用实际上就是操作原对象, 这与指针有相似之处, 但引用更安
    全和易用。
228     inline size_t memory(const Tensor& t) {
229         if (! t.has_storage()) {
230             return 0;
231         }
232         auto& storage = t.storage();
233         size_t res = storage.nbytes();
234         memory_sum += res;
235         memory_max = std::max(memory_max, res);
236         memory_count += 1;
237         return res;
238     }
239     ★ 重要函数
240     */
241     size_t memory(const Tensor& t);
242
243     /*模板结构体 RefCell, 表示一个带有引用计数的对象。*/
244     template <typename T>
245     struct RefCell final
246         : intrusive_ptr_target { // 继承自 intrusive_ptr_target, final
247         // 关键字表示该结构体不能被继承。
248         // value 是一个可变的模板类型 T 的成员变量, 即使在 const
249         // 成员函数中也可以修改。
250         mutable T value;
251         // release_resources 是一个重载的虚函数, 用于释放 RefCell 持有的资源。
252         void release_resources() final {
253             static_release_resources(value); // 释放 RefCell 持有的 value 资源。
254         }
255         // 构造函数, 接受一个 T 类型的常量引用 t, 并用它初始化成员变量 value。
256         RefCell(const T& t) : value(t) {}

```

```

257 };
258
259 /*模板类型别名 Ref，表示指向 RefCell<T> 类型对象的智能指针。
260 当使用 Ref<T> 类型的对象时，实际上是在操作一个指向 RefCell<T>
261 类型对象的智能指针，可以方便地管理 RefCell<T> 对象的生命周期和资源释放。
262 */
263 /*
264 Ref 是 intrusive_ptr<RefCell<T>> 的别名，即 Ref 是一个智能指针，指向 RefCell<T>
265 对象。这是一种 依赖 关系，因为 Ref 依赖于 RefCell<T>
266 类型，但它们之间并没有直接的成员变量关系。
267
268 使用了 using 关键字来创建一个类型别名 Ref。对于类型 T，Ref<T> 是
269 intrusive_ptr<RefCell<T>> 的别名。
270
271 通过继承 intrusive_ptr_target，RefCell 可以被 intrusive_ptr
272 管理，利用引用计数来管理对象的生命周期。
273 这样做的目的是为了 避免手动管理内存和资源，减少内存泄漏的风险。
274
275 开发者只需要使用 Ref<T> 就可以创建一个指向 RefCell<T>
276 的智能指针，而不需要每次都写 intrusive_ptr<RefCell<T>>。
277 */
278 template <typename T>
279 using Ref =
280     intrusive_ptr<RefCell<T>>; // 使用 intrusive_ptr 提供引用计数功能，自动管理
281                               // RefCell 对象的生命周期。
282
283 /*模板函数，用于显式地释放 intrusive_ptr 管理的资源。*/
284 template <typename T>
285 void static_release_resources(intrusive_ptr<T>& ptr) {
286     ptr.reset();
287 }
288
289 // 前向声明 CheckpointTensorCell 类，表示将在后续代码中定义该类。
290 class CheckpointTensorCell;
291
292 /**
293     这些类型别名用于定义 CheckpointTensorCell
294     类及其相关操作，提供更清晰的代码结构和可读性：
295
296     strong 和 weak 提供了对 CheckpointTensorCell
297     的强引用和弱引用，方便内存管理。 Tensors、rematerialize_function_t 和
298     mutate_function_t 用于张量操作和重计算机制。 time_t 和 duration_t 用于处理与
299     CheckpointTensorCell 相关的时间点和持续时间。
300 */
301
302 /*智能指针类型别名*/
303 /*强引用和弱引用是C++中智能指针的两种类型，它们分别代表不同的引用计数方式。

```



```

304 1.
305 强引用 (strong) : 强引用是一种普通的智能指针, 它会增加引用计数, 当引用计数减少到0时, 对
    象会被自动删除。
306     强引用主要用于在多个对象之间共享所有权, 确保对象在被使用时不会被意外删除。
307
308 2. 弱引用 (weak) : 弱引用不会增加引用计数, 因此不会影响对象的生命周期。
309     弱引用主要用于解决循环引用问题, 避免因为两个对象相互引用而导致的内存泄漏。弱引用需要
    在使用前转换为强引用, 以确保对象在被使用时不会被意外删除。
310 */
311 using strong = intrusive_ptr<CheckpointTensorCell>; // 强引用
312 using strongs = std::vector<strong>; // 定义为包含多个 strong 的
    // std::vector, 表示强引用的集合。
313
314 using weak = weak_intrusive_ptr<CheckpointTensorCell>; // 弱引用
315 using weaks =
    std::vector<weak>; // 定义为包含多个 weak 的 std::vector, 表示弱引用的集合。
316
317
318 /*张量和函数类型别名*/
319 using Tensors = std::vector<Tensor>; // 表示张量的集合
320 using rematerialize_function_t = std::function<Tensors(
321     const Tensors&>; // 表示重新计算张量的函数类型, 接受一个 Tensors
322     // 对象并返回一个 Tensors 对象。
323 using mutate_function_t =
324     std::function<void(const Tensors&>; // 表示修改张量的函数类型, 接受一个
325     // Tensors 对象, 不返回任何值。
326
327 /*时间相关类型别名*/
328 using time_t =
329     std::chrono::time_point<std::chrono::system_clock>; // 表示系统时钟的时间点
330 using duration_t =
331     std::chrono::system_clock::duration; // 表示系统时钟的时间持续时间
332
333 /*CheckpointInfo 结构体, 成员函数 cost 用于计算重计算的开销*/
334 struct CheckpointInfo {
335     // 成员变量, 表示计算成本, 用 duration_t 类型 (系统时钟的时间持续时间) 表示。
336     duration_t compute_cost;
337     // Floating Point instability? 浮点不稳定?
338     // 成员函数, 计算并返回一个 double
339     // 类型的值, 表示计算成本与内存和陈旧度的比率。
340
341     /**
342      * 被以下函数调用:
343      * - double AliasPool::cost
344      */
345     double cost(size_t memory, size_t staleness) const {
346         // 确保 memory 和 staleness 大于 0, 否则会触发错误检查。
347         TORCH_CHECK(memory > 0);
348         TORCH_CHECK(staleness > 0);

```



```

349         return compute_cost.count() /
350             static_cast<double>(
351                 memory * staleness); // compute_cost.count() 返回持续时间的计数值
352     }
353     // 构造函数，接受一个 duration_t 类型的参数 compute_cost，并初始化成员变量
354     // compute_cost。
355     CheckpointInfo(duration_t compute_cost) : compute_cost(compute_cost) {}
356 };
357
358 // ecn represent a evicted tensor group.
359 // it is a set of tensor that are evicted, and if two evicted tensor are input
360 // -> output to each other, they must be in an ecn. note: we try to support
361 // removal from ecn by subtracting compute_cost and memory. this will create
362 // suprious connection but that should be fine empircally. below is an example
363 // of a suprious connection: a -> b, a -> c a, b, c got evicted so belong to a
364 // single ecn. a got rematerialized. b, c still belong to a single ecn
    although
365 // there is no connection.
366
367 // ecn 表示驱逐的张量组 (evicted tensor group) 。
368 // 它是一组被驱逐的张量，如果两个驱逐的张量是输入 ->
369 // 输出关系，它们必须属于同一个 ecn。 注意：支持从 ecn 中移除张量，通过减去
370 // compute_cost 和内存，这可能会产生一些虚假的连接，但经验上应该问题不大。
371 // 下面是一个虚假连接的示例：
372 // a -> b, a -> c
373 // a、b、c 被驱逐，所以它们属于同一个 ecn。
374 // a 被重新计算后，b 和 c 仍然属于同一个 ecn，尽管没有直接连接。
375 using ecn_ptr = intrusive_ptr<EquivalentClassNode<CheckpointInfo>>;
376
377 /*Unsafe 的作用
378     在某些情况下，构造 Rematerializer
379     对象时需要特别小心，比如需要确保某些资源已经被初始化或处于特定状态
380
381     标签或标记类型：
382         Unsafe
383     结构体是一个空结构体，这种结构体通常用于作为标记或标签，表示某种特殊的状态或条件。
384     在 Rematerializer 的构造函数中，Unsafe
385     被用作一个参数。这样做的目的是强制要求调用者显式地传递一个 Unsafe 实例，
386     以表示他们了解构造函数的某种危险或不安全性。
387     这种模式可以增加代码的可读性和安全性，提醒开发者注意可能存在的特殊情况或风险。
388
389     防止默认构造：
390     使用 Unsafe 作为参数可以防止类的默认构造。在这种情况下，必须显式地传递一个
391     Unsafe 实例，这可以防止无意中创建对象， 从而避免潜在的错误或未初始化的状态。
392     这是一种设计模式，确保只有在特定上下文中（即调用者意识到不安全性并显式地传递
393     Unsafe 实例）才创建 Rematerializer 对象。
394 */

```

```

395 struct Unsafe {};
396
397 /*Rematerializer 结构体*/
398 // The rematerializer could be called to reinvoke an operator. Tensor point to
399 // remat which point to Tensor. To build the cycle remat support a default
400 // constructor, and allow you to fill in the member later. 可以调用
401 // rematerializer 来重新调用运算符。张量指向 remat (重新计算), remat
402 // 指向张量, 以形成循环。 remat 支持默认构造函数, 并允许稍后填充成员。
403 struct Rematerializer : intrusive_ptr_target {
404     // 成员变量
405     rematerialize_function_t func; // 用于重新计算张量的函数。
406
407     /*
408     强引用 (strong reference)
409     表示对象的生命周期由引用计数来管理。当引用计数增加时, 对象不会被销毁。
410     确保对象存活: 使用强引用意味着 Rematerializer 确保其输入
411     CheckpointTensorCell 对象在 Rematerializer 自身存活期间不会被销毁。这是因为
412     inputs 对象在计算或处理过程中需要被访问和使用。 依赖性: Rematerializer 对
413     inputs 有直接的依赖, 确保这些对象在其整个生命周期内是有效的。 如果 inputs
414     被销毁, Rematerializer 的操作将无法进行。
415
416     弱引用 (weak reference) 不会增加对象的引用计数, 不会阻止对象被销毁。
417     避免循环引用: 如果 outputs
418     使用强引用, 可能会导致循环引用问题, 导致内存泄漏。 例如, 如果
419     CheckpointTensorCell 也有对 Rematerializer 的强引用, 会导致相互之间无法释放。
420     非强依赖性: Rematerializer 可能不直接依赖 outputs 的生存。
421     弱引用表示 Rematerializer 不需要保证这些对象在其生命周期内一直存在。
422     outputs 对象可能在其他地方被强引用管理, Rematerializer
423     只需要引用它们但不需要控制它们的生命周期。 资源释放: 使用弱引用可以允许
424     CheckpointTensorCell 对象在不再需要时被释放, 即使 Rematerializer
425     还保有对它们的弱引用。这有助于更高效地管理内存和资源。
426
427     */
428     strongs inputs; // 输入张量的强引用集合。
429     weaks outputs; // 输出张量的弱引用集合。
430     duration_t compute_cost; // 重新计算的时间成本。
431     // when some output in here get evicted, they should belong to this ecn.
432     // A rematerializer have to track this, because when multiple output of a
433     // rematerializer get evicted, we only want to count the compute cost once.
434     // 当一些输出被驱逐时, 它们应该属于该 ecn。 rematerializer
435     // 跟踪这个, 因为当多个输出被驱逐时, 我们只想计算一次计算成本。
436     ecn_ptr ecn; // 驱逐张量组的智能指针。
437     // 构造函数
438     Rematerializer(
439         const Unsafe&,
440         const rematerialize_function_t& func, // 重新计算函数
441         const strongs& inputs, // 输入张量集合

```

```

442         duration_t compute_cost)
443     : // 重新计算的时间成本
444       func(func),
445       inputs(inputs),
446       compute_cost(compute_cost) {}
447 // 成员函数，用于释放资源
448 void release_resources() final {
449     func = rematerialize_function_t(); // 将 func 重置为空函数对象
450     inputs.clear(); // 清空 inputs 和 outputs 向量
451     outputs.clear();
452 }
453 // 成员函数声明
454 void remat();
455 ecn_ptr get_ecn();
456 CheckpointInfo get_cpi();
457 };
458
459 // Track all Tensor that share the same Storage.
460 // This is the atomic level of eviction - when evicting, everything here will
461 // get evicted. When an AliasPool is evicted, the Storage of the underlying
462 // tensor must be freed. Additionally, the AliasPool contain weak pointer to
463 // all
464 // children of tensors, in order to compute the score of evicting a Storage.
465 // 任何共享相同底层存储的张量都会被跟踪在同一个 AliasPool 中。
466 // 驱逐操作的最小单位是整个 AliasPool，即当驱逐操作发生时，AliasPool
467 // 中的所有张量都会被驱逐。 当 AliasPool
468 // 被驱逐时，底层张量的存储 (Storage) 必须被释放，以腾出内存。 AliasPool
469 // 包含所有子张量的弱指针 (weaks)，这些弱指针用于计算驱逐存储的分数。这意味着
470 // AliasPool 跟踪的不只是顶层张量，还包括所有派生或相关的张量。 AliasPool 中的
471 // weak tensors 和 weak neighbors 用于管理和跟踪这些张量及其关系。 lock_count
472 // 和 external_count
473 // 用于管理引用计数，确保在驱逐时这些计数为零，以避免驱逐正在使用的张量。
474 /*AliasPool
475  * 结构体，用于管理共享相同存储 (Storage) 的所有张量，在需要时执行驱逐操作，并计算驱逐成
476  * 本，确保系统在内存不足时可以有效释放资源。*/
477 struct AliasPool : intrusive_ptr_target {
478     weak tensors; // 弱指针集合，指向共享同一存储的张量。
479     weak neighbors; // 弱指针集合，指向邻居张量，用于计算驱逐存储的分数。
480
481     // 成员函数声明，返回邻居的 ecn_ptr 集合
482     std::set<ecn_ptr> neighbor_ecn();
483
484     // get() might hold some raw Tensor, rendering them unevictable.
485     // it is likely that get() will run out of memory, and when it does so, it
486     // will try to evict. so, it is crucial that we dont try to evict those
487     // tensors - doing so will not evict anything. lock_count count how many
488     time

```

```

486 // a tensor is referenced by get. get()
487 // 方法可能会引用某些原始张量，使它们无法被驱逐。 get()
488 // 方法可能会导致内存不足，并尝试驱逐。
489 // 因此，确保不尝试驱逐这些张量非常重要—这样做不会驱逐任何内容。
490 // lock_count 计算一个张量被 get 引用的次数。
491
492 size_t lock_count =
493     0; // 计数有多少次通过 get() 方法引用该张量，影响是否可驱逐。
494 size_t external_count = 0; // 外部引用计数。
495
496 // 增加或减少 lock_count，用于跟踪通过 get() 方法的引用。
497 void lock() {
498     ++lock_count;
499 }
500 void unlock() {
501     --lock_count;
502 }
503
504 intrusive_ptr<Rematerializer> head_remat; // 指向 Rematerializer 的智能指针。
505
506 // 返回是否可以驱逐，条件是 lock_count 为 0 且 head_remat 存在。
507 bool evictable() const {
508     return lock_count == 0 && head_remat;
509 }
510
511 // if it is not evictable it must not be evicted.
512 // 如果它不可被驱逐，则不得被驱逐。
513 bool is_evicted = false; // 标记该池是否已被驱逐。
514 size_t memory; // 该池占用的内存大小。
515 time_t last_used_time; // 上次使用时间。
516
517 // An aliaspool cant register itself to the checkpointpool - you have to do
it
518 // yourself. aliaspool 无法将自身注册到 checkpointpool - 您必须自己注册。
519 AliasPool(
520     const Unsafe&,
521     intrusive_ptr<Rematerializer> head_remat,
522     size_t memory)
523 : // 初始化 AliasPool 的成员变量。
524     head_remat(head_remat),
525     memory(memory),
526     last_used_time(std::chrono::system_clock::now()) {}
527
528 // if it is evicted, then hold the evicted tensor group.
529 // 如果被驱逐，则持有被驱逐的张量组。
530 ecn_ptr ecn; // 驱逐张量组的智能指针。
531

```

```

532 // 成员函数声明，计算驱逐成本，具体实现在.cpp
533 double cost(time_t current_time);
534 // 成员函数声明，驱逐操作，具体实现在.cpp
535 void evict();
536
537 // 增加 external_count 外部引用计数
538 void register_external() {
539     ++external_count;
540 }
541 // 减少 external_count，并在计数为 0 且符合条件时触发驱逐。
542 void release_external() {
543     // 每次调用 release_external 时，external_count 计数器减少。
544     --external_count;
545     // 如果 external_count 减少到 0 且 lock_count 为 0，调用 evict 方法将
546     // AliasPool 对象标记为已驱逐 (evicted)，释放相关资源。
547     if (external_count == 0) {
548         if (lock_count > 0) {
549             return;
550         }
551         TORCH_CHECK(lock_count == 0);
552         if (memory > 0 && (!ecn) && head_remat) {
553             evict();
554         }
555     }
556 }
557
558 // if it was evicted, refresh it. otherwise do nothing.
559 // have to check so, because when we rematerialize a single tensor in an
560 // aliaspool, we will set it to non-evicted, and when we rematerialize it's
561 // tensor they will also reset this. 如果 AliasPool
562 // 已被驱逐，则刷新其状态。否则，不执行任何操作。
563 // 这是必须的检查，因为当我们重新计算 AliasPool
564 // 中的单个张量时，我们将其设置为未驱逐状态，并且当我们重新计算其张量时，它们也会重置
    这个状态。
565 void set_not_evicted(const intrusive_ptr<AliasPool>& self);
566
567 // 释放资源，清空张量和邻居集合，重置 head_remat。
568 void release_resources() final {
569     tensors.clear();
570     neighbors.clear();
571     head_remat.reset();
572 }
573 };
574
575 /*CheckpointTensorCell 结构体，管理张量的存储、驱逐以及重计算。*/
576 struct CheckpointTensorCell : intrusive_ptr_target {
577     /*

```

```

578     t是一个std::unique_ptr<Tensor>类型的成员变量，用于存储实际的Tensor对象。
579     std::unique_ptr是一种智能指针，它提供了独占所有权的语义，即只有一个std::unique_ptr
    可以拥有所指向的对象，当std::unique_ptr被销毁时，它所指向的对象也会被自动销毁。
580     */
581     std::unique_ptr<Tensor> t; // 存储实际的 Tensor 对象。
582
583     bool defined = false; // 表示张量是否已定义。
584     bool is_undefined_tensor; // 表示张量是否为未定义状态。
585
586     DispatchKeySet key_set_; // 张量的调度键集合，用于标识张量的属性和操作。
587     // 获取调度键集合
588     DispatchKeySet key_set() const {
589         TORCH_CHECK(defined);
590         return key_set_;
591     }
592
593     caffe2::TypeMeta dtype_; // 数据类型
594     // 获取数据类型
595     caffe2::TypeMeta dtype() const {
596         TORCH_CHECK(defined);
597         return dtype_;
598     }
599
600     c10::optional<Device> optional_device_; // 张量所在的设备
601     // 获取设备
602     c10::optional<Device> optional_device() const {
603         TORCH_CHECK(defined);
604         return optional_device_;
605     }
606
607     // A Tensor is evictable iff it's AliasPool is evictable.
608     // A evictable tensor must have Rematerializer.
609     // 如果 AliasPool 是可驱逐的，则 Tensor 是可驱逐的。
610     // 可驱逐张量必须具有 Rematerializer。
611
612     /*
613     pool是一个intrusive_ptr<AliasPool>类型的成员变量，用于存储指向AliasPool对象的智能
    指针。
614     intrusive_ptr是一种侵入式智能指针，它需要对象本身提供一些额外的信息来管理引用计数。
615     创建一个指向AliasPool对象的智能指针。<>符号用于指定指针所指向的对象类型，这里表示pool
    是一个指向AliasPool类型的智能指针。
616     使用intrusive_ptr可以自动管理对象的生命周期，当没有任何指针指向该对象时，对象会被自动
    删除。
617     */
618
619     /*

```

620 *CheckpointTensorCell*: 这是一个结构体, 继承自 *intrusive_ptr_target*, 用于作为智能指针的目标类型。它包含了三个主要的成员变量: *t*、*pool*、和 *remat*。

621 *pool*: 这是 *CheckpointTensorCell* 中的一个成员变量, 其类型为 *intrusive_ptr<AliasPool>*。这表明 *pool* 是一个指向 *AliasPool* 对象的智能指针, 用于管理 *AliasPool* 对象的生命周期。

622 *AliasPool*: 这很可能是一个管理别名或资源池的类, 与 *CheckpointTensorCell* 共同工作以实现某些功能, 例如在神经网络训练中管理权重或状态的别名。

623 *intrusive_ptr*: 这是一种特殊类型的智能指针, 通常用于表达一种“强引用”关系, 即只要还有 *intrusive_ptr* 指向某个对象, 那个对象就不会被删除。

624

625 通过以上分析, 可以看出 *CheckpointTensorCell* 和 *pool* 之间的关系是典型的聚合关系, 其中 *CheckpointTensorCell* 聚合了一个 *AliasPool* 对象,

626 并通过 *intrusive_ptr* 智能指针来管理这个对象的生命周期。这种设计模式在面向对象编程中非常常见, 用于表示对象之间的所有权和生命周期管理关系。

627

628 总的来说, *CheckpointTensorCell* 通过包含一个指向 *AliasPool* 的智能指针 *pool*, 建立了与 *AliasPool* 的聚合关系。

629 这种设计使得 *CheckpointTensorCell* 可以方便地管理和使用 *AliasPool* 提供的服务或资源, 同时确保了 *AliasPool* 对象的生命周期得到妥善管理。

630 **/*

631 *intrusive_ptr<AliasPool> pool;*

632 */**

633 *remat* 是一个 *intrusive_ptr<Rematerializer>* 类型的成员变量, 用于存储指向 *Rematerializer* 对象的智能指针。

634 *CheckpointTensorCell* 和 *Rematerializer* 之间的关系是一种组合关系。

635 组合关系表示一个类包含另一个类的实例, 并且这个实例的生命周期由包含它的类管理。

636 *remat* 是 *CheckpointTensorCell* 类的一个组成部分, 这意味着 *CheckpointTensorCell*

637 拥有并负责管理 *remat* 的生命周期。

638 **/*

639 *intrusive_ptr<Rematerializer> remat;*

640

641 */***

642 ** 被以下函数调用:*

643 ** - void AliasPool::evict*

644 **/*

645 *// 将张量重置 (驱逐)*

646 *void evict() {*

647 *TORCH_CHECK(remat);*

648 *t.reset();*

649 *}*

650

651 *// 将一个新的 Tensor 填充到 CheckpointTensorCell 中。实现在.cpp*

652 *void fill(const Tensor& t);*

653

654 *// 构造函数, 使用张量和内存池初始化*

655 *explicit CheckpointTensorCell(*

656 *const Tensor& t,*


```

657         const intrusive_ptr<AliasPool>& pool)
658         : pool(pool) {
659     fill(t);
660 }
661 // 构造函数，使用张量、内存池和重计算器初始化
662 explicit CheckpointTensorCell(
663     const Tensor& t,
664     const intrusive_ptr<AliasPool>& pool,
665     const intrusive_ptr<Rematerializer>& remat)
666     : pool(pool), remat(remat) {
667     fill(t);
668 }
669
670 // 获取内存大小，用来记录日志用的，没啥实际用处
671 size_t memory() {
672     TORCH_CHECK(defined);
673     return pool->memory;
674 }
675
676 /**
677  * 被以下函数调用：
678  * - void pin
679  * - Tensor uncheckpoint
680  * - Tensor decheckpoint
681  * - Tensors uncheckpoint
682  */
683 // 获取当前的 Tensor 对象。如果 Tensor
684 // 对象不存在（例如被驱逐了），则通过重计算（rematerialization）机制来重新生成它。
685 Tensor get() {
686     // 如果当前对象的 Tensor 指针 t 为 nullptr，表示 Tensor
687     // 对象不存在或已被驱逐。
688     if (!t) {
689         // 确保重计算函数 remat 存在
690         TORCH_CHECK(remat);
691         // 执行重计算过程，重新生成 Tensor 对象。remat 是一个 Rematerializer
692         // 对象，负责根据原始输入和重计算函数重新计算 Tensor。
693         remat->remat();
694     }
695     // 确保 Tensor 对象在重计算后已生成。如果 t
696     // 仍然为空，会抛出异常，表明重计算过程失败。
697     TORCH_CHECK(t);
698     // 确保张量的调度键集合中不包含
699     // Checkpoint。这是为了确保张量在返回前没有错误的调度键。
700     TORCH_CHECK(!t->key_set().has(DispatchKey::Checkpoint));
701     // 更新 AliasPool 的 last_used_time 属性为当前时间，记录该 Tensor
702     // 的最后使用时间。
703     pool->last_used_time = std::chrono::system_clock::now();

```

```

704     return *t; // 返回 Tensor 对象的引用
705 }
706
707 /**
708  * 被以下函数调用：
709  * - void pin
710  * - void clear_checkpointpool
711 */
712 // 固定张量，使其不会被驱逐。通过确保张量存在，并清除重计算相关的状态，使得张量不会再
// 通过重计算机制被释放或驱逐。
713 void pin() {
714     // 调用 get() 方法，确保当前 Tensor 对象存在。
715     // 如果 Tensor 不存在，get() 方法会通过调用 remat->remat()
716     // 进行重计算来重新生成 Tensor。确保张量存在是后续操作的前提条件。
717     get();
718     // 清除内存池的重计算状态
719     // 清除 AliasPool 对象中的 head_remat 指针。head_remat 是一个指向
720     // Rematerializer 的智能指针，表示该内存池的头部重计算器。
721     // 重置这个指针意味着当前内存池中的张量不再与重计算器关联，从而防止张量通过重计算机
// 制被驱逐。
722     pool->head_remat.reset();
723     // 同样地，remat 是一个指向 Rematerializer
724     // 的智能指针，表示当前张量的重计算器。
725     // 重置这个指针意味着当前张量不再与重计算器关联，从而防止张量通过重计算机制被释放。
726     remat.reset();
727 }
728
729 // 释放 CheckpointTensorCell
730 // 对象持有的资源。该函数在对象被销毁或不再需要时调用，以确保所有与之相关的资源（特别
// 是内存资源）被正确释放。
731 void release_resources() final {
732     // 释放 Tensor 对象占用的内存。这是通过调用 std::unique_ptr 的 reset()
733     // 方法实现的，它会删除当前指向的对象并将指针置为 nullptr。
734     t.reset();
735     // 减少 AliasPool 对象的引用计数，并在引用计数归零时释放 AliasPool
736     // 对象。这是通过调用 intrusive_ptr 的 reset() 方法实现的。
737     pool.reset();
738     // 减少 Rematerializer 对象的引用计数，并在引用计数归零时释放 Rematerializer
739     // 对象。这是通过调用 intrusive_ptr 的 reset() 方法实现的
740     remat.reset();
741 }
742 };
743
744 // An external reference. Each strong will have at most one external
// reference.
745 // By keeping such an invariant, whenever an external reference die, we know

```

```

746 // that the underlying strong is only used internally. Thus, when it die we
    can
747 // apply optimization like banishing/infinite staleness. We keep this
    invariant
748 // by only allowing CheckpointTensorImpl to make new External, when new
749 // CheckpointTensorImpl is constructed. 外部引用。每个 strong
750 // 最多有一个外部引用。
751 // 通过保持这样的不变量，每当外部引用死亡时，我们就知道底层的 strong
752 // 仅在内部使用。 因此，当它死亡时，我们可以应用诸如 banishing/infinite
753 // staleness 之类的优化。 当构造新的 CheckpointTensorImpl 时，我们只允许
754 // CheckpointTensorImpl 创建新的外部来保持这个不变性。
755 /*External 结构体表示对一个 CheckpointTensorCell 的外部引用。每个
756 * strong (强引用) 最多有一个外部引用，确保在外部引用消亡时，可以进行优化操作。*/
757 struct External : intrusive_ptr_target {
758     // 构造函数
759     // 接受一个强引用 value 并将其赋值给成员变量 value。
760     External(const strong& value) : value(value) {
761         // 调用 value->pool->register_external() 注册该外部引用。
762         value->pool->register_external();
763     }
764     // 接受一个 Tensor 对象 value。
765     External(const Tensor& value)
766         : // 创建一个新的 AliasPool，并通过调用另一个构造函数 External(const
767           // strong&), 将新创建的 strong 引用传递过去。
768           External(strong::make(
769               value,
770               intrusive_ptr<AliasPool>::make(
771                   Unsafe(),
772                   intrusive_ptr<Rematerializer>(),
773                   memory(value)))) {}
774     // 接受一个 Tensor 对象 value, 一个 AliasPool 指针 pool, 和一个 Rematerializer
775     // 指针 remat。
776     External(
777         const Tensor& value,
778         const intrusive_ptr<AliasPool>& pool,
779         const intrusive_ptr<Rematerializer>& remat)
780         : // 创建一个新的 strong 引用，并通过调用另一个构造函数 External(const
781           // strong&), 将新创建的 strong 引用传递过去。
782           External(strong::make(value, pool, remat)) {}
783     strong value; // value 是一个强引用，指向一个 CheckpointTensorCell 对象。
784     // 成员函数声明，释放 External 对象所持有的资源。这个函数在 External
785     // 对象被销毁时被调用。实现在.cpp
786     void release_resources() override;
787 };
788
789 // 内联函数，用于将传入的 DispatchKeySet 对象进行修改，添加
790 // DispatchKey::Checkpoint 键。

```

```

791 inline DispatchKeySet convert_key_set(const DispatchKeySet& t) {
792     // 检查 DispatchKeySet 是否包含 Checkpoint 键, 使用 CHECK 宏来断言 t 中不包含
793     // DispatchKey::Checkpoint 键。 如果 t 中已经包含了 Checkpoint
794     // 键, 这一断言将失败, 程序将中止。这确保了 Checkpoint 键不会被重复添加。
795     CHECK(!t.has(DispatchKey::Checkpoint));
796     // 在 t 的副本上添加 DispatchKey::Checkpoint 键, 并返回一个新的 DispatchKeySet
797     // 对象。 DispatchKeySet 类中的 add
798     // 方法返回一个包含新键的副本, 而不会修改原始对象 t。
799     auto ret = t.add(DispatchKey::Checkpoint);
800     return ret;
801 }
802
803 // 继承自 TensorImpl, 实现了一个具有动态张量重计算机制的张量实现。通过引用
804 // External
805 // 对象来管理实际的张量数据, 并提供了丰富的接口来支持张量的属性获取和操作。
806 // 它通过 convert_key_set 函数确保 Checkpoint
807 // 键的添加, 使用唯一标识符和引用计数来管理张量的生命周期和资源。
808 struct CheckpointTensorImpl : TensorImpl {
809     int id = gen_counter(); // id 是一个独特的标识符, 通过 gen_counter() 生成。
810     static int counter; // counter 是一个静态整型变量, 用于生成唯一的 id。
811     // 通过递增 counter, 生成唯一的标识符。
812     static int gen_counter() {
813         return counter++;
814     }
815     // 返回以 "x" 开头, 后跟唯一 id 的字符串, 生成标识符名称。
816     std::string counter_name() const {
817         return std::string("x") + std::to_string(id);
818     }
819
820     // 持有 External 对象的引用, 用于管理实际张量数据。
821     /*
822     Ref<intrusive_ptr<External>> 展开为
823     intrusive_ptr<RefCell<intrusive_ptr<External>>>。 RefCell<T> 是模板类, 因此
824     RefCell<intrusive_ptr<External>> 是一个具体的 RefCell 类实例, 其成员 value
825     的类型是 intrusive_ptr<External>。
826     intrusive_ptr<RefCell<intrusive_ptr<External>>> 是一个智能指针,
827     指向一个包含 intrusive_ptr<External> 类型值的 RefCell 对象。
828     */
829     Ref<intrusive_ptr<External>> ref;
830
831     // 函数声明, 释放 CheckpointTensorImpl 对象持有的资源。
832     void release_resources() final;
833
834     // 构造函数1,
835     explicit CheckpointTensorImpl(const Ref<intrusive_ptr<External>>& ref)
836         : // convert_key_set 用于添加 Checkpoint 键。设置 dtype 和
837           // optional_device。

```

```

838         TensorImpl(
839             convert_key_set(ref->value->value->key_set()),
840             ref->value->value->dtype(),
841             ref->value->value->optional_device()),
842         ref(ref) {
843     // 如果 key_set 包含 Autograd 键, 则设置 requires_grad 为 true。
844     if (key_set().has(DispatchKey::Autograd)) {
845         set_requires_grad(true);
846     }
847 }
848 // 构造函数2, 使用 Ref<intrusive_ptr<External>> 构造函数来初始化对象。
849 explicit CheckpointTensorImpl(const intrusive_ptr<External>& e)
850     : CheckpointTensorImpl(Ref<intrusive_ptr<External>>::make(e)) {}
851 // 构造函数3, 通过直接传入 Tensor 对象来初始化。
852 explicit CheckpointTensorImpl(const Tensor& t);
853
854 // 用于创建和变更张量的静态函数, 实现在.cpp
855 static Tensors make(
856     const std::string& name,
857     const rematerialize_function_t& remat,
858     const Tensors& inputs);
859 // mutate_idx indicate which of the inputs will get mutated.
860 // mutate_idx 指示哪些输入将发生变化。
861 static void mutate(
862     const std::string& name,
863     const mutate_function_t& mutate,
864     const Tensors& inputs,
865     const std::vector<size_t>& mutate_idx);
866
867 // 用于浅复制和分离张量对象。实现在.cpp
868 intrusive_ptr<TensorImpl> shallow_copy_and_detach(
869     const VariableVersion& version_counter,
870     bool allow_tensor_metadata_change) const override;
871 void shallow_copy_from(const c10::intrusive_ptr<TensorImpl>& impl) override;
872
873 // 通过引用 External
874 // 对象来获取张量的各类属性, 如维度、元素数量、大小和步长等。
875 int64_t dim() const override {
876     return ref->value->value->get().dim();
877 }
878 int64_t numel() const override {
879     return ref->value->value->get().numel();
880 }
881 IntArrayRef sizes() const override {
882     return ref->value->value->get().sizes();
883 }
884 int64_t size(int64_t d) const override {

```

```

885     return ref->value->value->get().size(d);
886 }
887 IntArrayRef strides() const override {
888     return ref->value->value->get().strides();
889 }
890 int64_t stride(int64_t d) const override {
891     return ref->value->value->get().stride(d);
892 }
893 bool has_storage() const override {
894     return false;
895 }
896 };
897
898 // CheckpointPool keep a list of AliasPool, and search over them to choose the
899 // best one to evict. CheckpointPool 保留一份 AliasPool
900 // 列表, 并搜索它们以选择最好的一个进行驱逐。
901 /*CheckpointPool 是一个管理 AliasPool 和 External
902 * 对象的容器, 它提供了一些方法用于自动逐出和内存管理。*/
903 struct CheckpointPool {
904     // 成员变量
905     std::vector<weak_intrusive_ptr<AliasPool>>
906         aps; // 存储 AliasPool 的弱引用指针列表。防止这些对象被自动删除。
907     std::vector<weak_intrusive_ptr<External>>
908         exts; // 存储 External 的弱引用指针列表。防止这些对象被自动删除。
909     std::random_device rd; // 用于生成随机数种子。
910     std::mt19937 gen =
911         std::mt19937(rd()); // 标准的梅森旋转随机数生成器, 使用 rd 作为种子。
912     bool sample_tensors = true; // 是否在逐出循环中对池进行平方根采样。
913     bool ignore_small_tensors = true; // 是否忽略小于平均张量大小1%的张量。
914     bool has_memory_budget = false; // 是否有内存预算限制。
915     long memory_budget; // 内存预算的大小。
916
917     // 成员函数
918     void evict(); // 逐出一些张量以释放内存
919     void auto_evict(); // 在内存使用超过预算时自动逐出张量
920     void clear_checkpointpool(); // 只定义, 没有实现。
921     void add(const intrusive_ptr<AliasPool>&); // 将一个 AliasPool 对象添加到 aps
922                                           // 列表中
923
924     // 构造函数
925     CheckpointPool();
926 };
927
928 /*通过 dynamic_cast 将 Tensor 的底层实现转换为 CheckpointTensorImpl*
929 * 类型, 并检查转换是否成功。*/
930 inline CheckpointTensorImpl* get_cpti(const Tensor& t) {
931     auto* cpti = dynamic_cast<CheckpointTensorImpl*>(t.unsafeGetTensorImpl());

```

```

932     TORCH_CHECK(cpti != nullptr); // 如果转换失败，TORCH_CHECK 会触发异常。
933     return cpti; // 返回 CheckpointTensorImpl* 指针。
934 }
935
936 /*获取 Tensor 对象的 CheckpointTensorImpl 实现并返回其引用 (ref)*/
937 inline Ref<intrusive_ptr<External>> cell_from_tensor(const Tensor& t) {
938     return get_cpti(t)->ref;
939 }
940
941 } // namespace at

```

CheckpointTensorImpl.cpp

```

1  #include <ATen/CheckpointTensorImpl.h>
2  #include <ATen/Logger.h>
3  #include <c10/cuda/CUDACachingAllocator.h>
4
5  #include <chrono> // 处理时间
6  #include <string>
7  #include <random>
8  #include <cmath>
9
10 namespace at {
11
12 // 定义类型别名
13 using Clock = std::chrono::high_resolution_clock; // 使用高分辨率时钟。
14 using Time = Clock::time_point; // 表示时间点。
15 using Duration = Clock::duration; // 表示时间间隔。
16 using FinalTime = std::chrono::nanoseconds; // 以纳秒为单位的时间间隔。
17
18 struct PerfStats;
19
20 /*Timer 结构体，用于测量代码片段的执行时间。*/
21 struct Timer {
22     std::string name; // 计时器名称。
23     Time start; // 计时器开始时间。
24     // 构造函数：初始化计时器名称和开始时间。
25     Timer(std::string name, Time start) : name(name), start(start) {}
26     Timer() {}
27     // 析构函数：在计时器生命周期结束时记录时间间隔。
28     ~Timer();
29 };
30
31 constexpr bool stats = true;

```



```

32
33  /*PerfStats 结构体用于记录和输出性能统计信息，包括函数调用的次数和持续时间。它包含计时
    器功能，可以跟踪不同代码片段的执行时间。*/
34  struct PerfStats {
35      // 定义了 TimerStats 类型，表示单个计时器的统计信息。
36      //  std::string: 计时器名称。
37      //  Time: 计时器开始时间。
38      //  Time: 计时器结束时间。
39      //  Duration: 计时器持续时间。
40      using TimerStats = std::tuple<std::string, Time, Time, Duration>;
41
42      // 成员变量，功能：记录性能统计信息的成员变量。
43      //  start: 记录 PerfStats 实例创建的时间。
44      //  calls: 记录每个计时器名称的调用次数。
45      //  timers: 记录所有计时器的统计信息。
46      Time start;
47      std::unordered_map<std::string, int> calls;
48      std::vector<PerfStats::TimerStats> timers;
49
50      // 构造函数，初始化 PerfStats 实例。
51      //  start: 设置为当前时间。
52      //  calls: 初始化为空的 std::unordered_map。
53      //  timers: 初始化为空的 std::vector。
54      PerfStats() : start(Clock::now()), calls(0), timers() {}
55
56      // 开始计时并记录调用次数。
57      /*Timer track(std::string name) {
58          // 在 calls 中查找计时器名称 name，如果找到则调用次数加一，否则插入新记录。
59          if (stats) {
60              auto it = this->calls.find(name);
61              if (it != this->calls.end()) {
62                  it->second += 1;
63              } else {
64                  this->calls.insert({name, 0});
65              }
66              // 返回一个新的 Timer 对象。
67              return Timer(name, Clock::now());
68          }
69          return Timer();
70      }*/
71
72      void track(const char*) { } // 一个空的跟踪方法，可能是预留用于扩展功能。
73
74      // 析构函数，在 PerfStats 实例销毁时输出性能统计信息。
75      ~PerfStats() {
76          // 判断 stats 是否启用：如果未启用性能统计，则直接返回。
77          if (!stats) { return; }

```

```

78
79     // 从 timers 向量的第一个元素中获取起始时间 start。注意，这里假定 timers 向量至少
    包含一个元素。
80     auto start = std::get<1>(this->timers[0]);
81     // 获取当前时间 now。
82     auto now = Clock::now();
83
84     std::cout << "All done. Here are some perf stats fresh off the presses."
    << std::endl; // 全做完了。以下是一些新鲜出炉的性能统计数据。
85
86     // 定义 durations 哈希表，用于存储每个计时器名称对应的总持续时间。
87     std::unordered_map<std::string, Duration> durations;
88     // 计算 PerfStats 实例创建到当前的总时间 total。
89     Duration total = now - this->start;
90
91     // For now simple strategy, count up all the time taken by each "tagged
    call site". 对于现在的简单策略，计算每个“标记的调用站点”所花费的所有时间。
92     // 遍历 timers 向量中的每个计时器记录。
93     // 获取计时器名称 name 和持续时间 duration。
94     // 在 durations 哈希表中查找 name:
95     // 如果找到，则累加 duration 到现有值。
96     // 如果未找到，则插入新的 name 和 duration。
97     for (auto timer : timers) {
98         auto name = std::get<0>(timer);
99         Duration duration = std::get<3>(timer);
100        auto it = durations.find(name);
101        if (it != durations.end()) {
102            it->second += duration;
103        } else {
104            durations.insert({name, duration});
105        }
106    }
107
108    std::vector<std::pair<std::string, Duration>> data;
109
110    // Convert the durations
111    // 转换持续时间，将 durations 哈希表转换为 data 向量，以便排序。
112    for (auto d : durations) {
113        // auto duration = std::chrono::duration_cast<FinalTime>(d.second); 自动
    持续时间
114        data.push_back(d);
115    }
116
117    // 按时间降序排序 data。
118    std::sort(data.begin(), data.end(),
119        [](const std::pair<std::string, Duration> & a, const
    std::pair<std::string, Duration> & b) -> bool {

```

```

120         return a.second > b.second;
121     });
122
123     // 遍历排序后的 data 向量，输出每个计时器的名称、调用次数、总时间和占总时间的百分比。
124     for (auto d : data) {
125         // 将每个计时器的持续时间 duration 和总持续时间 total_duration 转换为
        FinalTime 单位（纳秒）。
126         auto duration = std::chrono::duration_cast<FinalTime>(d.second);
127         auto total_duration = std::chrono::duration_cast<FinalTime>(total);
128
129         // 计算并存储每个计时器的持续时间占总时间的百分比 percentage。
130         double percentage =
            ((double)duration.count())/((double)total_duration.count()) * 100;
131
132         // 在 calls 哈希表中查找计时器名称 d.first 对应的调用次数 call_count，并检查其
        存在性。
133         auto call_count = this->calls.find(d.first);
134         TORCH_CHECK(call_count != this->calls.end());
135
136         // 输出每个计时器的名称、调用次数、持续时间和百分比。
137         std::cout << "CallSite: " << d.first << " CallCount: " << call_count-
            >second << " Cost: " << duration.count() << "ns" << " (% " << percentage << " )"
            << std::endl;
138     }
139 }
140 };
141
142 // 创建一个全局的 PerfStats 实例，用于在整个程序中收集性能统计信息。
143 // 使用 PerfStats 默认构造函数初始化 STATS 实例。
144 static PerfStats STATS = PerfStats();
145
146 /*全局变量*/
147 size_t memory_sum = 0;    // 累计内存使用情况。
148 size_t memory_max = 0;    // 记录内存使用的峰值。
149 size_t memory_count = 0;  // 计算内存占用的张量数量。
150
151 /*重置内存统计数据。*/
152 void reset_memory_stat() {
153     memory_sum = 0;
154     memory_max = 0;
155     memory_count = 0;
156 }
157
158 /**
159  * 被以下函数调用：
160  * - MakeRawResult make_raw

```

```

161  */
162  /*内联函数，计算一个 Tensor 对象的内存占用，并更新全局统计数据*/
163  inline size_t memory(const Tensor& t) {
164      // 检查张量是否有存储。如果没有存储，返回 0 表示内存占用为零。
165      if (! t.has_storage()) {
166          return 0;
167      }
168      // 获取张量的存储对象引用 storage。
169      auto& storage = t.storage();
170      // 获取存储对象的字节数，即内存占用大小，并将结果存储在 res 中。
171      size_t res = storage.nbytes();
172      // 更新内存统计数据
173      memory_sum += res; // 累加所有计算的内存占用。
174      memory_max = std::max(memory_max, res); // 更新记录的最大内存占用。
175      memory_count += 1; // 记录计算内存占用的张量数量。
176      return res;
177  }
178
179  /*Timer 析构函数：在 Timer 对象销毁时记录时间间隔，并将其添加到全局 PerfStats 实例的
timers 向量中。*/
180  Timer::~~Timer() {
181      // 获取当前时间 now。
182      Time now = Clock::now();
183      // 计算从 start 到 now 的时间间隔 elapsed。
184      Duration elapsed = now - start;
185      // 创建一个包含计时器名称、开始时间、结束时间和时间间隔的元组 stats。
186      PerfStats::TimerStats stats = { name , start, now, elapsed };
187      // 将 stats 添加到全局 PerfStats 实例 STATS 的 timers 向量中。
188      STATS.timers.push_back(stats);
189  }
190
191  bool use_log_ = false; // 控制日志记录功能的启用状态。
192  bool use_profile_ = false; // 控制性能分析功能的启用状态。
193  long base_compute_time_ = 0; // 记录基础计算时间。
194  long remat_compute_time_ = 0; // 记录重计算时间。
195  long search_time_ = 0; // 记录搜索时间。
196  long cost_time_ = 0; // 记录成本时间。
197
198  // 声明一个 CheckpointPool 实例，名为 pool。这将作为一个全局变量，表示整个检查点池。
199  CheckpointPool pool;
200
201  /**
202   * 被以下函数调用：
203   * - void AliasPool::set_not_evicted
204   * - MakeRawResult make_raw
205   */
206  /*将一个 AliasPool 添加到 CheckpointPool 中*/

```

```

207 void CheckpointPool::add(const intrusive_ptr<AliasPool>& p) { //
    intrusive_ptr 是一种智能指针类型，提供引用计数管理。
208     /**
209     * 检查是否应将 AliasPool p 添加到 aps 向量中。
210     * p->memory > 0: 确保 AliasPool p 所占用的内存大于 0。
211     * memory_count == 0: 如果没有记录任何内存使用信息，直接允许添加。
212     * !ignore_small_tensors: 如果设置不忽略小张量，直接允许添加。
213     * 如果 p 所占用的内存大于平均张量大小的 1%，允许添加。
214     * memory_sum 是所有张量内存的总和，memory_count 是张量的数量。
215     */
216     if (p->memory > 0 && (memory_count == 0 || !ignore_small_tensors || p->
memory >= 0.01 * double(memory_sum/memory_count))) {
217         // 将 p 的弱引用 (weak_intrusive_ptr<AliasPool>) 添加到 aps 向量中。
218         // 使用 weak_intrusive_ptr 可以避免循环引用导致的内存泄漏。
219         aps.push_back(weak_intrusive_ptr<AliasPool>(p));
220     }
221 }
222
223 /**
224 * 被以下函数调用：
225 * - void CheckpointPool::auto_evict
226 */
227 /*获取当前设备 (GPU 0) 的内存使用情况。*/
228 long current_memory() {
229     STATS.track("current_memory");
230     // 获取设备 0 的内存统计信息。
231     auto device_stat = c10::cuda::CUDACachingAllocator::getDeviceStats(0);
232     // 返回当前分配的字节数
233     return device_stat.allocated_bytes[0].current;
234 }
235
236 /**
237 * 被以下函数调用：
238 * - void Rematerializer::remat
239 * - MakeRawResult make_raw
240 */
241 /*在 CheckpointPool 达到或超过内存预算时自动进行张量的逐出 (eviction)，以确保内存使用保持在预算范围内。*/
242 void CheckpointPool::auto_evict() {
243     STATS.track("CheckpointPool::auto_evict");
244     // 检查是否有内存预算限制。
245     if (has_memory_budget) {
246         while (current_memory() > memory_budget) {
247             evict(); // 逐出 (删除) 一些张量或其他数据，以释放内存。
248         }
249     }
250 }

```

```

251
252 /**
253  * 被以下函数调用:
254  * - void CheckpointPool::auto_evict
255  */
256 /*从 CheckpointPool 中选择一个合适的 AliasPool 进行逐出操作, 以释放内存资源。*/
257 void CheckpointPool::evict() {
258     // 记录函数开始执行的时间。
259     time_t pre = std::chrono::system_clock::now();
260
261     STATS.track("CheckpointPool::evict");
262
263     // 确保 aps 列表中至少有一个 AliasPool 可以被逐出。
264     TORCH_CHECK(aps.size() > 0);
265
266     // shrunk: either something has been evicted or the pools have gotten
    smaller
267     bool shrunk = false;           // 要么有什么东西被逐出了, 要么池变小了
268     int evict_idx = -1;           // 要逐出的索引。初始化逐出索引为 -1, 表示尚未确定要
    逐出的对象。
269     double evict_cost = INFINITY; // 初始化逐出成本为无穷大, 用于后续比较选择成本最小
    的对象。
270
271     // 获取当前时间, 用于计算逐出成本。
272     time_t current_time = std::chrono::system_clock::now();
273
274     // 定义一个 lambda 函数, 用于将无效的 AliasPool 从 aps 列表中移除。该函数通过将目标
    元素与列表最后一个元素交换, 然后弹出最后一个元素实现移除操作。
275     auto remove_from_aps = [&](size_t i) {
276         aps[i] = aps[aps.size() - 1]; // 将要移除的元素与最
    后一个元素交换
277         aps.pop_back();               // 弹出最后一个元素
278     };
279     // 定义一个均匀分布, 用于随机采样 aps 列表中的元素。采样的范围是 1 到 aps 列表大小的
    平方根。
280     std::uniform_int_distribution<> distrib(1, 1 * std::max(1, static_cast<int>
    (std::sqrt(aps.size()))));
281
282     // sampling a random independent subset of all evictable tensors to find
    the cheapest tensor to evict.
283     // 对所有可驱逐张量的随机独立子集进行采样, 以找到要驱逐的最便宜的张量。
284
285     // 遍历 aps 列表中的每个 AliasPool。
286     for (size_t i = 0; i < aps.size(); i) {
287         auto cannot_evict = [&]() {
288             shrunk = true;           // 标记已缩减
289             remove_from_aps(i); // 移除无效的 aps

```

```

290         };
291         // 获取弱引用的强引用。
292         auto ap_strong = aps[i].lock();
293         // 如果强引用未定义，调用 cannot_evict 函数标记已缩减并移除无效的 AliasPool。
294         if (!ap_strong.defined()) {
295             cannot_evict();
296         }
297         // 如果 AliasPool 设置了 ecn，代表该 tensor 已经被逐出了，也不能逐出，调用
cannot_evict 函数。
298         else if (ap_strong->ecn) {
299             cannot_evict();
300         }
301         else {
302             // 如果 AliasPool 可逐出，计算其逐出成本并与当前最小成本比较。如果成本更低，更新
最小成本和逐出索引。
303             if (ap_strong->evictable()) {
304                 double cost = ap_strong->cost(current_time); // 计算逐出成本
305                 if (cost < evict_cost) {
306                     evict_cost = cost; // 更新逐出成本
307                     evict_idx = i;      // 更新逐出索引
308                 }
309             }
310             // 根据 sample_tensors 标志决定是否进行随机采样。
311             if (sample_tensors) {
312                 i += distrib(gen); // 随机跳过一些元素
313             } else {
314                 i += 1;           // 逐个检查元素
315             }
316         }
317     }
318     // 如果没有找到适合逐出的 AliasPool，确保 shrunk 标志为真。
319     if (evict_idx == -1) {
320         TORCH_CHECK(shrunk);
321     } else { // 如果找到了逐出索引，定义并调用 evict_from_idx 函数执行逐出操作，并从
aps 列表中移除逐出的 AliasPool。
322         auto evict_from_idx = [&](size_t idx) {
323             auto ap_strong = aps[idx].lock(); // 获取要逐出的
强引用
324             TORCH_CHECK(ap_strong.defined());
325             ap_strong->evict(); // 执行逐出操作
326             remove_from_aps(evict_idx); // 从 aps 列表
中移除
327         };
328         evict_from_idx(evict_idx);
329     }
330     // 记录函数结束执行的时间。
331     time_t post = std::chrono::system_clock::now();

```



```

332     // 更新搜索时间，累加本次操作的耗时。
333     search_time_ += (post - pre).count();
334 }
335
336 /*构造函数*/
337 CheckpointPool::CheckpointPool() { }
338
339 namespace native {
340
341 /**
342  * 被以下函数调用：
343  * - Tensor try_checkpoint
344  */
345 /*将张量 t 转换为检查点化张量，并返回新的检查点化张量。*/
346 Tensor checkpoint(const Tensor& t) {
347     // 记录 checkpoint 函数的调用次数或执行时间。
348     STATS.track("checkpoint");
349     // 使用 t 创建一个 CheckpointTensorImpl 对象 cpti。
350     // intrusive_ptr<CheckpointTensorImpl>::make(t) 是工厂方法，返回一个
    intrusive_ptr，指向新创建的 CheckpointTensorImpl 对象。
351     auto cpti = intrusive_ptr<CheckpointTensorImpl>::make(t);
352     // 如果启用了日志功能，则记录日志。
353     if (use_log_) {
354         // 记录常量日志。
355         DTRLogConstant(cpti->counter_name());
356         // 记录内存日志。
357         DTRLogMemory(cpti->counter_name(), cpti->ref->value->value->memory());
358     }
359     // 返回一个新的 Tensor，其实为 CheckpointTensorImpl 对象 cpti。
360     return Tensor(cpti);
361 }
362
363 /*解封 CheckpointTensorImpl，返回原始的 Tensor。*/
364 Tensor uncheckpoint(const Tensor& t) {
365     STATS.track("uncheckpoint");
366     // 将 t 的底层实现动态转换为 CheckpointTensorImpl 类型的指针。
367     auto* cpti = dynamic_cast<CheckpointTensorImpl*>(t.unsafeGetTensorImpl());
368     TORCH_CHECK(cpti != nullptr);
369     // 获取 cpti 的引用 (ref)，再从中获取 External 对象的值 (value)，然后继续获取嵌套
    的 Tensor 对象，最终返回该张量。
370     return cpti->ref->value->value->get();
371 }
372
373 /*确保该张量不会被驱逐出内存或被垃圾回收。*/
374 void pin(Tensor& t) {
375     STATS.track("pin");
376     auto* cpti = dynamic_cast<CheckpointTensorImpl*>(t.unsafeGetTensorImpl());

```

```

377     TORCH_CHECK(cpti != nullptr);
378     // 获取 cpti 的引用 (ref) , 再从中获取 External 对象的值 (value) , 然后继续获取嵌套
    的 Tensor 对象, 最终调用其 pin 方法。
379     cpti->ref->value->value->pin();
380 }
381
382 /*如果输入张量是 CheckpointTensorImpl 类型, 则返回其原始张量; 否则返回原始输入张量。
    */
383 Tensor decheckpoint(const Tensor& t) {
384     STATS.track("decheckpoint");
385     auto* cpti = dynamic_cast<CheckpointTensorImpl*>(t.unsafeGetTensorImpl());
386     // 如果 cpti 非空, 获取 cpti 的引用 (ref) , 再从中获取 External 对象的值
    (value) , 然后继续获取嵌套的 Tensor 对象, 最终返回该张量。
387     // 如果 cpti 为空, 表示 t 不是一个 CheckpointTensorImpl, 直接返回原始张量 t。
388     return cpti ? cpti->ref->value->value->get() : t;
389 }
390
391 /**
392  * 被以下函数调用:
393  * - Tensor try_checkpoint
394  */
395 /*检查张量 t 是否已经是检查点化张量。*/
396 bool is_checkpoint(const Tensor& t) {
397     // 记录 is_checkpoint 函数的调用次数或执行时间。
398     STATS.track("is_checkpoint");
399     // 通过 dynamic_cast 将张量 t 的实现类转换为 CheckpointTensorImpl 类型的指针
    cpti。
400     auto* cpti = dynamic_cast<CheckpointTensorImpl*>(t.unsafeGetTensorImpl());
401     // 如果转换成功, 即 cpti 不为 nullptr, 则表示 t 是一个检查点化张量, 返回 true。
402     return cpti != nullptr;
403 }
404
405 /**
406  * 被以下函数调用:
407  * - Tensors try_checkpoint
408  */
409 /*尝试将张量 t 转换为检查点化张量。*/
410 Tensor try_checkpoint(const Tensor& t) {
411     // 记录 try_checkpoint 函数的调用次数或执行时间。
412     STATS.track("try_checkpoint");
413     // 如果 t 已经是检查点化的张量, 则直接返回 t。否则, 调用 checkpoint(t) 对张量 t 进
    行检查点化, 并返回检查点化后的张量。
414     return is_checkpoint(t) ? t : checkpoint(t);
415 }
416
417 /*创建一个新的日志文件, 并将其设置为当前的输出日志文件。通过传入一个字符串来确定日志文件
    的名称。*/

```

```

418 void new_log(std::string str) {
419     DTRLogger::logger().out =
420     std::ofstream(DTRLogger::logger().get_filename(str));
421 }
422
423 /*在日志文件中记录注释信息。根据全局配置，可以选择记录为 JSON 格式或简单的文本格式。*/
424 void annotate_log(std::string str) {
425     if (!use_log_) { return; }
426     if (log_json) {
427         json j;
428         j[INSTRUCTION] = "ANNOTATE";
429         j[ANNOTATION] = str;
430         DTRLogger::logger().log(j.dump());
431     } else {
432         DTRLogger::logger().log("# " + str);
433     }
434 }
435
436 /*启用或禁用日志记录功能，通过修改全局变量 use_log_ 来控制。*/
437 void toggle_log(bool b) {
438     use_log_ = b;
439 }
440
441 /*清空 pool.exts 列表，并确保所有仍然存在的 External 对象及其相关的
CheckpointTensorCell 被固定在内存中，防止它们被逐出或释放。*/
442 void clear_checkpointpool() {
443     // likely 是一个优化提示，告知编译器在大多数情况下，这个条件是 true。
444     // 循环检查 pool.exts 列表是否为空，如果不为空则进入循环体。
445     while (likely(!pool.exts.empty())) {
446         // pool.exts.back() 取得 exts 向量的最后一个元素，这是一个
weak_intrusive_ptr<External> 类型的对象。
447         // lock() 方法尝试从这个弱引用获得一个强引用 intrusive_ptr<External>。如果对象仍
然存在（没有被销毁），则 lock() 成功，返回一个有效的强引用 intrusive_ptr<External>，
并赋值给变量 e。
448         // 如果对象已经被销毁，lock() 返回 nullptr，此时 e 为 false，if 条件不成立，跳
过 if 语句体。
449         if (auto e = pool.exts.back().lock()) {
450             // 当 lock() 成功时，e 持有一个有效的 intrusive_ptr<External>。
451             // e->value 获取 External 对象的 value 成员，这是一个 strong 类型的指针，指
向 CheckpointTensorCell。
452             // 调用 pin() 方法。这意味着将相应的 CheckpointTensorCell 固定在内存中，防止其
被逐出或释放。
453             // pin() 方法的实现（在 CheckpointTensorCell 中）会通过调用 get() 方法确保张
量数据已经被加载并驻留在内存中，并重置 remat 和 pool->head_remat，这有助于释放相关的资
源。
454             e->value->pin();
455         }

```

```
455     // 从 exts 向量的末尾移除最后一个元素。
456     pool.exts.pop_back();
457 }
458 }
459
460 /*取消内存预算限制。*/
461 void unset_memory_budget() {
462     pool.has_memory_budget = false;
463 }
464
465 /*设置内存预算限制。*/
466 void set_memory_budget(long budget) {
467     pool.memory_budget = budget;
468     pool.has_memory_budget = true;
469 }
470
471 /*切换是否在内存管理中采样张量。*/
472 void toggle_sampling(bool sample) {
473     pool.sample_tensors = sample;
474 }
475
476 /*切换是否忽略小张量。*/
477 void toggle_ignore_small_tensors(bool ignore) {
478     pool.ignore_small_tensors = ignore;
479 }
480
481 /*将所有相关的时间计数器变量重置为 0。*/
482 void reset_profile() {
483     base_compute_time_ = 0;
484     remat_compute_time_ = 0;
485     search_time_ = 0;
486     cost_time_ = 0;
487 }
488
489 /*切换性能分析功能的启用状态。*/
490 void toggle_profile(bool profile) {
491     use_profile_ = profile;
492 }
493
494 /*计算总的计算时间*/
495 long compute_time() {
496     return base_compute_time() + remat_compute_time();
497 }
498
499 /*获取成本时间。*/
500 long cost_time() {
501     return cost_time_;
```

```

502 }
503
504 /*获取搜索时间。*/
505 long search_time() {
506     return search_time_;
507 }
508
509 /*获取重计算时间。*/
510 long remat_compute_time() {
511     return remat_compute_time_;
512 }
513
514 /*获取基础计算时间。*/
515 long base_compute_time() {
516     return base_compute_time_;
517 }
518
519 /*计算循环时间。*/
520 long loop_time() {
521     return search_time() - cost_time();
522 }
523
524 }
525
526 /*内联版本的 uncheckpoint 函数，该函数接受一个 strong 类型的参数并返回对应的 Tensor
对象。*/
527 [[inline]] // [[inline]]: 这个修饰符提示编译器尽量将函数内联展开，以减少函数调用的开
销。内联展开意味着在编译过程中，函数调用将被替换为函数体本身，从而避免了调用函数的开销。
528 Tensor uncheckpoint(const strong& input) { // strong 类型的常量引用，指向一个
CheckpointTensorCell 对象。
529     return input->get(); // 返回 Tensor 对象，即从 input 指向的
CheckpointTensorCell 对象中提取的张量。
530 }
531
532 /**
533  * 被以下函数调用：
534  * - void Rematerializer::remat
535  * - MakeRawResult make_raw
536  */
537 /*将一组 strong 指针指向的 CheckpointTensorCell 对象转化为实际的 Tensor 对象。*/
538 Tensors uncheckpoint(const strongs& inputs) {
539     STATS.track("uncheckpoint");
540     // 创建一个 Tensors 类型的向量 ret
541     Tensors ret;
542     // 预分配与输入向量 inputs 相同的大小，以减少内存分配次数，提高性能。
543     ret.reserve(inputs.size());

```

```

544 // 对 inputs 中的每个 strong 指针（指向 CheckpointTensorCell 对象）进行遍历，并调
    用 input->get() 提取 Tensor 对象，将其添加到 ret 向量中。
545 for (const strong& input : inputs) {
546     // inlined manually 手动内联
547     ret.push_back(input->get());
548 }
549 return ret;
550 };
551
552 /**
553  * 被以下函数调用：
554  * - Tensors CheckpointTensorImpl::make
555  * - void CheckpointTensorImpl::mutate
556 */
557 /*尝试对输入的张量进行检查点化，返回检查点化后的张量列表。*/
558 Tensors try_checkpoint(const Tensors& inputs) { // inputs: 输入的张量列表
559     STATS.track("try_checkpoint"); // 记录 try_checkpoint 函数的调用次数或执行时
    间。
560     Tensors ret; // 定义一个空的张量列表，用于存储检查点化后的张量。
561     ret.reserve(inputs.size()); // 预先分配 ret 列表的容量，以避免后续多次分配内
    存。
562     // 遍历输入张量并进行检查点化
563     for (const Tensor& input : inputs) {
564         ret.push_back(at::native::try_checkpoint(input));
565     }
566     return ret;
567 }
568
569 /**
570  * 被以下函数调用：
571  * - double AliasPool::cost
572  * - void AliasPool::evict
573 */
574 /*合并两个 CheckpointInfo 实例，并返回一个新的 CheckpointInfo 实例，其
    compute_cost 是两个输入实例的 compute_cost 之和。*/
575 CheckpointInfo merge_cpi(CheckpointInfo l, CheckpointInfo r) {
576     // 记录 merge_cpi 方法的调用统计信息，用于性能分析和调优。
577     STATS.track("merge_cpi");
578     // 创建一个新的 CheckpointInfo 实例，其 compute_cost 为输入参数 l 和 r 的
    compute_cost 之和。
579     return CheckpointInfo(l.compute_cost + r.compute_cost);
580 }
581
582 /**
583  * 被以下函数调用：
584  * - void CheckpointPool::evict
585 */

```

```

586  /*驱逐当前 AliasPool*/
587  void AliasPool::evict() {
588      STATS.track("AliasPool::evict");
589      // 检查并获取 ecn
590      TORCH_CHECK(!ecn);
591      ecn = head_remat->get_ecn();
592      // 合并邻居 ecn
593      auto ecns = neighbor_ecn();
594      for (const auto& necn : ecns) {
595          // 合并邻居 ecn 的 CheckpointInfo, 合并的目的是聚合计算成本, 以便在决策时能考虑到
          // 更大范围的计算代价。
596          merge<CheckpointInfo>(merge_cpi, ecn, necn);
597      }
598      // 验证条件
599      TORCH_CHECK(memory > 0);
600      TORCH_CHECK(lock_count == 0);
601      TORCH_CHECK(!is_evicted);
602      // 更新驱逐状态并驱逐张量
603      is_evicted = true;
604      for (const weak& w : tensors) {
605          if (auto cell = w.lock()) {
606              cell->evict();
607          }
608      }
609  }
610
611  /**
612   * 被以下函数调用:
613   * - void CheckpointPool::evict
614   */
615  /*计算驱逐成本。*/
616  double AliasPool::cost(time_t current_time) {
617      // 记录开始时间
618      time_t pre = std::chrono::system_clock::now();
619      // 获取 CheckpointInfo
620      auto cpi = head_remat->get_cpi();
621      // 合并邻居 ecn 的 CheckpointInfo
622      auto ecns = neighbor_ecn();
623      for (const auto& necn : ecns) {
624          cpi = merge_cpi(cpi, get_t(necn));
625      }
626      // 计算成本
627      auto ret = cpi.cost(memory, (current_time - last_used_time).count());
628      // 记录结束时间并更新 cost_time_
629      time_t post = std::chrono::system_clock::now();
630      cost_time_ += (post - pre).count();
631      return ret; // 返回成本

```

```

632 }
633
634 /*释放 External 对象所持有的资源。*/
635 void External::release_resources() {
636     // pool 是一个指向 AliasPool 对象的智能指针，而 release_external 方法用于处理外部
    引用的释放逻辑。
637     // release_external 方法主要负责减少 external_count 计数器，并在必要时触发 evict
    操作。
638     value->pool->release_external();
639     // 将 value 指针重置为 nullptr，意味着 External 不再持有对 CheckpointTensorCell
    对象的引用。
640     // 重置指针是为了确保资源能够正确释放，并避免潜在的内存泄漏问题。
641     value.reset();
642 }
643
644 /**
645  * 被以下函数调用：
646  * - Tensor CheckpointTensorCell::get
647  */
648 /*执行重新计算操作，处理输入张量，调用重计算函数，更新输出张量，处理锁定和解锁。*/
649 void Rematerializer::remat() {
650     // TODO: refactor using RAII for exception safety.
651     // 锁定输入池：遍历 inputs，对每个输入张量的池 (pool) 进行加锁，以确保线程安全。
652     for (const strong& s : inputs) {
653         s->pool->lock();
654     }
655     // 取消检查点，调用 uncheckpoint 函数，从输入张量集合中恢复张量。
656     Tensors ts = uncheckpoint(inputs);
657     // 记录重新计算前后的时间，并调用 func(ts) 执行重新计算。
658     time_t pre = std::chrono::system_clock::now();
659     auto ret = func(ts); // 重计算的结果
660     time_t post = std::chrono::system_clock::now();
661     // 调用 pool.auto_evict() 进行自动驱逐。
662     pool.auto_evict();
663     // 更新重新计算的时间成本
664     remat_compute_time_ += (post - pre).count();
665     // 确保重新计算的结果数量与预期输出数量一致。
666     TORCH_CHECK(ret.size() == outputs.size());
667     // 遍历输出集合，如果输出张量的弱引用有效，填充计算结果
668     for (size_t i = 0; i < outputs.size(); ++i) {
669         if (auto output_cell = outputs[i].lock()) {
670             output_cell->fill(ret[i]); // 把重计算出的tensor填充到checkpointcell里
671             // 在这个地方加 recompute_times
672         }
673     }
674     // 重置 ecn
675     ecn.reset();

```



```

676 // 解锁输入池
677 for (const strong& s : inputs) {
678     s->pool->unlock();
679 }
680 }
681
682 /**
683  * 被以下函数调用:
684  * - void AliasPool::evict
685  */
686 /*获取或创建一个驱逐张量组 ecn。*/
687 ecn_ptr Rematerializer::get_ecn() {
688     // 如果 ecn 为空, 创建一个新的 ecn_ptr, 其包含 CheckpointInfo 对象, 初始化计算成本。
689     /**
690      * get_ecn函数确保每个Rematerializer对象都有一个与之关联的CheckpointInfo节点
691      * 这个函数的目的是获取或创建与当前Rematerializer对象关联的CheckpointInfo节点
692      * (ecn_ptr是一个智能指针, 指向EquivalentClassNode<CheckpointInfo>类型的对象)。
693      * CheckpointInfo节点代表一组被逐出的张量, 如果两个被逐出的张量是输入-输出关系, 它们
694      * 必须属于同一个CheckpointInfo节点集合。
695      * 函数首先检查ecn成员变量是否已经指向了一个CheckpointInfo节点 (即ecn是否非空)。
696      * 如果ecn为空, 说明这是第一次调用此函数, 需要创建一个新的CheckpointInfo节点。
697      * 使用make函数和当前Rematerializer的compute_cost作为参数来创建一个新的
698      * CheckpointInfo节点, 并将其地址赋给ecn。
699      * 最后, 函数返回ecn的智能指针。
700      */
701     if (!ecn) {
702         ecn = ecn_ptr::make(CheckpointInfo(compute_cost));
703     }
704     return ecn;
705 }
706 /**
707  * 被以下函数调用:
708  * - double AliasPool::cost
709  * get_cpi函数则提供了一种获取当前重计算成本的方式, 无论是独立的成本 (当ecn为空时) 还是已经合并到某个集合中的成本 (当ecn非空时)
710     这个函数的目的是获取当前Rematerializer对象的CheckpointInfo对象。
711     函数首先检查ecn是否为空。如果ecn不为空, 说明已经存在一个CheckpointInfo节点, 此时返回一个新的CheckpointInfo对象, 其compute_cost设置为duration_t(0)。这可能表示该Rematerializer的计算成本已经被计入到ecn所代表的集合中, 因此这里返回一个成本为零的对象。
712     如果ecn为空, 说明没有创建对应的CheckpointInfo节点, 此时直接使用当前Rematerializer的compute_cost来创建并返回一个新的CheckpointInfo对象。
713     */
714 /*获取检查点信息, 根据 ecn 是否存在返回不同的计算成本。*/
715 CheckpointInfo Rematerializer::get_cpi() {
716     // 如果 ecn 非空, 返回计算成本为 0 的 CheckpointInfo;
717     // 否则, 返回包含当前计算成本的 CheckpointInfo。

```

```

715     return CheckpointInfo(ecn ? duration_t(0) : compute_cost);
716 }
717
718 /**
719  * 被以下函数调用:
720  * - void AliasPool::evict
721  * - double AliasPool::cost
722  */
723 /*获取当前 AliasPool 的邻居中所有有效的 ecn (evicted tensor group) 指针集合。*/
724 std::set<ecn_ptr> AliasPool::neighbor_ecn() {
725     // 记录 neighbor_ecn 方法调用的统计信息。
726     STATS.track("AliasPool::neighbor_ecn");
727     // 初始化指针集合
728     std::set<ecn_ptr> ptr_set;
729     // 遍历 neighbors, 锁定有效的 weak_ptr。
730     int size = neighbors.size();
731     for (size_t i = 0; i < size; i) {
732         if (auto cptc = neighbors[i].lock()) {
733             // 如果邻居中存在有效的 ecn, 将其插入 ptr_set。
734             if (cptc->pool->ecn) {
735                 ptr_set.insert(cptc->pool->ecn);
736             }
737             ++i;
738         } else { // 如果邻居无效, 将其移除并调整数组大小。
739             neighbors[i] = neighbors[size - 1];
740             size = size - 1;
741         }
742     }
743     if (size < neighbors.size()) {
744         neighbors.erase(neighbors.begin() + size);
745     }
746     return ptr_set; // 返回有效的 ecn 指针集合。
747 }
748
749 /**
750  * 被以下函数调用:
751  * - void CheckpointTensorCell::fill
752  */
753 /*如果当前 AliasPool 已被驱逐, 重置其状态为未驱逐。
754 将一个已经被逐出的AliasPool对象的状态重新设置为未逐出 ("not evicted")
755 */
756 void AliasPool::set_not_evicted(const intrusive_ptr<AliasPool>& self) {
757     if (unlikely(is_evicted)) { // 如果当前池已被驱逐
758         STATS.track("AliasPool::set_not_evicted(inside)");
759         is_evicted = false; // 重置驱逐状态
760         // 更新 ecn 信息
761         if (ecn) {

```

```

762     TORCH_CHECK(head_remat);
763     auto cpi = get_t(ecn);
764     update_t(ecn, CheckpointInfo(cpi.compute_cost - head_remat-
>compute_cost));
765     ecn.reset();
766 }
767 // 重新注册到 pool
768 pool.add(self);
769 }
770 }
771
772 /**
773  * 被以下函数调用:
774  * - void Rematerializer::remat
775  */
776 /*将一个新的 Tensor 填充到 CheckpointTensorCell 中。包含了一系列逻辑, 以确保
Tensor 的状态和相关属性正确设置。*/
777 // fill 方法在第一次调用时保存张量的内容, 并将其标记为已定义。
778 void CheckpointTensorCell::fill(const Tensor& t) {
779     // 使用 STATS.track 来记录函数调用的统计数据, 便于后续性能分析和调试。
780     STATS.track("CheckpointTensorCell::fill");
781     // 只有在当前对象的张量指针 this->t 未定义时, 才会创建一个新的 Tensor 对象。
782     if (!(this->t)) {
783         // 使用 t.detach() 创建一个独立于原始计算图的张量, 防止在重计算时干扰原始计算图。
784         this->t = std::make_unique<Tensor>(t.detach());
785         // 确保内存池未被驱逐
786         pool->set_not_evicted(pool);
787         // 只有在当前 CheckpointTensorCell 未定义时, 才会设置张量的属性。
788         if (!defined) {
789             defined = true; // 表示张量已定义。
790             is_undefined_tensor = !t.defined(); // 表示张量是否未定义 (例如, 空张
量)。
791             key_set_ = t.key_set(); // 保存张量的调度键集合
792             if (t.requires_grad()) {
793                 key_set_ = key_set_.add(DispatchKey::Autograd);
794             }
795             dtype_ = t.dtype(); // 保存张量的数据类型
796             optional_device_ = t.optional_device(); // 保存张量的设备信息
797         }
798     }
799 }
800
801 /*创建并返回一个新的 CheckpointTensorImpl 对象, 其内部状态是当前对象的浅拷贝。*/
802 intrusive_ptr<TensorImpl> CheckpointTensorImpl::shallow_copy_and_detach(const
VariableVersion& version_counter, // 变量版本计数器, 用于跟踪张量的版本变化 (在
这个函数中未使用)。

```

```

803                                                                 bool
allow_tensor_metadata_change) const { // 布尔值，指示是否允许更改张量的元数据（在
    这个函数中未使用）。
804     // 创建新对象：使用 intrusive_ptr<CheckpointTensorImpl>::make(ref) 创建一个新
    的 CheckpointTensorImpl 实例，传递当前对象的引用 ref 进行浅拷贝。
805     auto ret = intrusive_ptr<CheckpointTensorImpl>::make(ref);
806     // 日志记录：如果启用了日志功能 (use_log_)，调用 DTRLogCopy 记录拷贝操作。
807     if (use_log_) {
808         DTRLogCopy(ret->counter_name(), counter_name());
809     }
810     // 返回新的 CheckpointTensorImpl 对象的 intrusive_ptr<TensorImpl>。
811     return ret;
812 }
813
814 /*从另一个 CheckpointTensorImpl 对象浅拷贝其内部状态到当前对象。*/
815 void CheckpointTensorImpl::shallow_copy_from(const
c10::intrusive_ptr<TensorImpl>& impl) { // impl: 指向另一个 TensorImpl 对象的
    intrusive_ptr
816     STATS.track("CheckpointTensorCell::shallow_copy_from");
817     // 使用 TORCH_CHECK 确保 impl 具有 DispatchKey::CheckpointTensorId。
818     TORCH_CHECK(impl->key_set().has(DispatchKey::CheckpointTensorId));
819     // 将 impl 转换为 CheckpointTensorImpl 类型，确保转换成功。
820     auto* cpti = dynamic_cast<CheckpointTensorImpl*>(impl.get());
821     TORCH_CHECK(cpti != nullptr);
822     // 将 impl 的引用对象的值浅拷贝到当前对象的引用对象中。
823     ref->value = cpti->ref->value;
824     // 如果启用了日志功能 (use_log_)，调用 DTRLogCopyFrom 记录拷贝操作。
825     if (use_log_) {
826         DTRLogCopyFrom(counter_name(), cpti->counter_name());
827     }
828 }
829
830 /*静态成员变量 counter 定义及初始化*/
831 // 静态成员变量属于类本身，而不是类的某个实例。所有 CheckpointTensorImpl 对象共享同一
    一个 counter 变量。
832 // counter 用于生成 CheckpointTensorImpl 对象的唯一标识符。确保每个
    CheckpointTensorImpl 对象都拥有一个唯一的 id。
833 int CheckpointTensorImpl::counter = 0;
834
835 /*检查两个 Tensor 是否是别名关系*/
836 bool is_alias(const Tensor& l, const Tensor& r) {
837     // 首先检查两个 Tensor 是否已定义。
838     // 然后调用 Tensor 的 is_alias_of 方法来检查它们是否指向同一个底层数据。
839     // 如果两个 Tensor 的 Storage 对象共享相同的 storage_impl_，则它们被认为是别名关
    系。
840     return l.defined() && r.defined() && l.is_alias_of(r);
841 }

```

```

842
843 /**
844  * 被以下函数调用：
845  * - MakeRawResult make_raw
846 */
847 // return an index for alias. we dont care which one because they all lead to
    the same alias pool.
848 // return -1 for no alias.
849 // 返回别名的索引。我们不在乎是哪一个，因为它们都通向同一个别名池。
850 // 如果没有别名，则返回 -1。
851 /*在一组 Tensor 中找到与给定 Tensor 是别名关系的索引*/
852 int get_alias(const Tensors& ts, const Tensor& t) {
853     if (t.defined()) { // 检查 t 是否已定义。
854         // 遍历 ts 中的每个 Tensor，如果发现某个 Tensor 与 t 是别名关系，返回它的索引。
855         for (size_t i = 0; i < ts.size(); ++i) {
856             if (ts[i].defined() && t.is_alias_of(ts[i])) {
857                 return i;
858             }
859         }
860     }
861     // 如果未找到任何别名关系，返回 -1。
862     return -1;
863 }
864
865 /*MakeRawResult 结构体，用于封装重计算过程的结果*/
866 struct MakeRawResult {
867     // 存储重计算后的输出张量，每个张量被封装在一个 intrusive_ptr<External> 对象中，确
    保智能指针管理的对象生命周期。
868     std::vector<intrusive_ptr<External>> outputs;
869     // 存储输出张量与输入张量的别名关系。对于每个输出张量，如果它与某个输入张量共享内存
    (即别名)，则 aliases 向量中的相应元素保存该输入张量的索引。如果没有别名，则元素值为
    -1。
870     std::vector<int> aliases;
871     // 表示计算过程所耗费的时间。
872     duration_t time;
873     // 存储一个指向重计算器对象的智能指针。Rematerializer 对象负责管理重计算操作及其相关
    状态。
874     intrusive_ptr<Rematerializer> rematerializer;
875 };
876
877 /**
878  * 被以下函数调用：
879  * - MakeRawResult make_raw
880 */
881 /*通过将两个 AliasPool 标记为邻居，可以在内存管理和逐出时考虑它们之间的关联关系。*/
882 void add_neighbor(const strong& l, const strong& r) { // strong 是指向
    External 对象的智能指针。

```

```

883 // 将 l 和 r 各自的 AliasPool 中的 neighbors 列表互相添加对方的弱引用。
884 // weak 是弱引用，防止循环引用导致的内存泄漏。
885 l->pool->neighbors.push_back(weak(r));
886 r->pool->neighbors.push_back(weak(l));
887 }
888
889 /**
890  * 被以下函数调用：
891  * - Tensors CheckpointTensorImpl::make
892  * - void CheckpointTensorImpl::mutate
893  */
894 // remat take a single vector of tensors, while there are two vector, one
// storing nonconstants and one storing constants.
895 // the constants are small and they will not be considered for eviction.
896 // however, we have to stitch the two vectors together to pass it in remat.
897 // the size_t in constants decide the location to stitch them in, while
// input_values fill in the rest.
898 // remat 采用单个张量向量，而有两个向量，一个存储非常量，一个存储常量。
899 // 常量很小，不会被考虑驱逐。
900 // 但是，我们必须将两个向量拼接在一起才能将其传递到 remat 中。
901 // 常量中的 size_t 决定缝合它们的位置，而 input_values 则填写其余部分。
902 /*处理重新生成 (rematerialize) 张量，并返回一个 MakeRawResult 结构，该结构包含生成的
张量及其相关信息。*/
903 MakeRawResult make_raw(const rematerialize_function_t& remat_f, // 一个函数，定
义了如何重新生成张量。
904                        const strongs& inputs) { // 一个 strongs 类型的容器，包含
了输入张量的强引用（即 strong 对象）。
905     STATS.track("make_raw");
906
907     // 遍历输入张量的强引用，并对其内存池进行加锁，防止在重新生成过程中内存被其他操作修
改。
908     for (const strong& s : inputs) {
909         s->pool->lock();
910     }
911
912     // 调用 uncheckpoint(inputs) 将强引用的 strong 对象转换为实际的 Tensor 对象。
913     // uncheckpoint 函数通过 input->get() 获取实际的张量数据。
914     Tensors raw_inputs = uncheckpoint(inputs);
915
916     // 记录当前时间 pre 以测量重计算过程的耗时。
917     time_t pre = std::chrono::system_clock::now();
918     // 使用 remat_f 重计算原始输入张量 raw_inputs，得到重计算后的输出张量
raw_outputs。
919     auto raw_outputs = remat_f(raw_inputs);
920     // 记录重计算后的时间 post。
921     time_t post = std::chrono::system_clock::now();
922

```

```

923 // 自动驱逐可能不再需要的资源，确保内存使用在预算内。
924 pool.auto_evict();
925
926 // 计算重计算时间
927 base_compute_time_ += (post - pre).count();
928
929 // 创建输出张量、别名和弱引用的容器。
930 std::vector<intrusive_ptr<External>> outputs;
931 std::vector<int> aliases;
932 weaks weak_outputs;
933 // 使用 rematerializer 创建一个重物化器对象，并将其与输入张量和计算时间关联。
934 auto remat = intrusive_ptr<Rematerializer>::make(Unsafe(), remat_f, inputs,
post - pre); // post - pre 也就是刚才的 base_compute_time_
935
936 for (const Tensor& t : raw_outputs) { // 遍历 raw_outputs，对每个重新生成的输出张量进行处理。
937     intrusive_ptr<AliasPool> alias_pool;
938     // 调用 get_alias 检查输出张量是否与输入张量中的某个张量共享相同的底层存储（即是否为别名）。
939     int alias = get_alias(raw_inputs, t);
940     // 如果不是别名 (alias == -1)，则创建一个新的 AliasPool 对象，并将其添加到内存池中。
941     if (alias == -1) {
942         auto m = memory(t);
943         alias_pool = intrusive_ptr<AliasPool>::make(Unsafe(), remat, m); // 所以，一个 tensor 的 Rematerializer 是计算产生这个 tensor 的。
944         pool.add(alias_pool);
945     }
946     // 如果是别名，则复用已有的 AliasPool 对象，并累加计算成本。
947     else {
948         alias_pool = inputs[alias]->pool;
949         if (alias_pool->head_remat) {
950             alias_pool->head_remat->compute_cost += (post - pre);
951         }
952     }
953
954     // 使用生成的张量、别名池和重物化器对象创建一个 External 对象。
955     auto e = intrusive_ptr<External>::make(t, alias_pool, remat);
956
957     // 将 External 对象的弱引用添加到内存池的 exts 容器中。
958     pool.exts.push_back(weak_intrusive_ptr<External>(e));
959     // 将 External 对象的值的弱引用添加到别名池的张量列表中。
960     alias_pool->tensors.push_back(weak(e->value));
961
962     // 将 External 对象添加到输出列表 outputs。
963     outputs.push_back(e);
964     // 将别名信息添加到别名列表 aliases。

```

```

965     aliases.push_back(alias);
966     // 将输出张量的弱引用添加到弱引用列表 weak_outputs。
967     weak_outputs.push_back(weak(outputs.back()->value));
968 }
969
970 // 更新重计算器的输出
971 remat->outputs = weak_outputs;
972
973 // 遍历输入和输出张量，对每一对非别名关系的张量，调用 add_neighbor 更新其邻居关系。
974 for (size_t i = 0; i < inputs.size(); ++i) {
975     for (size_t j = 0; j < outputs.size(); ++j) {
976         // is_alias 检查 raw_inputs 中的 Tensor 与 raw_outputs 中的 Tensor 是否是别名关系，以决定是否需要调用 add_neighbor 将它们添加为邻居。
977         if (!is_alias(raw_inputs[i], raw_outputs[j])) {
978             // add_neighbor 将没有别名关系的输入和输出 Tensor 的 AliasPool 互相添加为邻居，以便在内存管理和逐出时考虑它们之间的关联关系。
979             add_neighbor(inputs[i], outputs[j]->value);
980         }
981     }
982 }
983
984 // 遍历输入张量的强引用，并对其内存池进行解锁。
985 for (const strong& s : inputs) {
986     s->pool->unlock();
987 }
988 // 返回一个 MakeRawResult 结构，包含输出张量、别名信息、时间信息和重计算器 (rematerializer) 对象。
989 return {outputs, aliases, post - pre, remat};
990 }
991
992 /*将持续时间 (duration_t) 转换为字符串格式的纳秒数。*/
993 std::string from_time(duration_t t) {
994     // 以字符串形式返回持续时间的纳秒数。
995     return std::to_string(std::chrono::nanoseconds(t).count());
996 }
997
998 /**
999  * 被 aten/src/ATen/native/Checkpoint.cpp 中的函数调用
1000 */
1001 /*创建新的检查点张量 (Checkpoint Tensor) */
1002 Tensors CheckpointTensorImpl::make(const std::string& name,
1003                                     const rematerialize_function_t& remat,
1004                                     const Tensors& inputs) {
1005     STATS.track("CheckPointTensorImpl::make");
1006
1007     // 使用 try_checkpoint 函数将输入张量进行检查点化，并获取检查点化后的张量数目。
1008     Tensors checkpointed_inputs = try_checkpoint(inputs);

```



```
1009     auto input_size = checkpointed_inputs.size();
1010
1011     // 初始化用于存储输入张量值和日志记录参数的容器。
1012     strongs input_values;
1013     input_values.reserve(input_size);
1014     std::vector<std::string> args;
1015     args.reserve(input_size);
1016
1017     // 遍历检查点化后的输入张量，提取每个张量的值并存储在 input_values 中。如果启用了日
    志记录功能，将每个张量的计数器名称添加到 args 中。
1018     for (const Tensor& t: checkpointed_inputs) {
1019         auto* cpti = dynamic_cast<CheckpointTensorImpl*>(t.unsafeGetTensorImpl());
1020         TORCH_CHECK(cpti);
1021         input_values.push_back(cpti->ref->value->value);
1022         if (use_log_) {
1023             args.push_back(cpti->counter_name());
1024         }
1025     }
1026
1027     // 调用 make_raw 函数进行重新生成，获取输出张量及其相关信息。
1028     auto ret = make_raw(remat, input_values);
1029
1030     Tensors tensors;
1031     tensors.reserve(ret.outputs.size());
1032
1033     // 遍历 make_raw 返回的输出张量，将其转换为 CheckpointTensorImpl 类型并存储在
    tensors 容器中。
1034     for (const auto& t: ret.outputs) {
1035         auto cp = Tensor(intrusive_ptr<CheckpointTensorImpl>::make(t));
1036         tensors.push_back(cp);
1037     }
1038
1039     // 如果启用了日志记录，则记录操作
1040     if (use_log_) {
1041         std::vector<std::string> res;
1042         res.reserve(ret.outputs.size());
1043
1044         // 获取每个输出张量的计数器名称并存储在 res 容器中。
1045         for (const auto& tensor : tensors) {
1046             res.push_back(get_cpti(tensor)->counter_name());
1047         }
1048
1049         // 调用 DTRLogCall 记录函数调用信息。
1050         DTRLogCall(res, name, args, from_time(ret.time));
1051
1052         // 遍历输出张量，记录每个张量的内存使用情况和别名信息。
1053         for (size_t i = 0; i < tensors.size(); ++i) {
```

```

1054     Tensor t = tensors[i];
1055     auto cpti = get_cpti(t);
1056     DTRLogMemory(cpti->counter_name(), cpti->ref->value->value->memory());
1057     DTRLogAlias(cpti->counter_name(), ret.aliases[i]);
1058 }
1059 }
1060
1061 return tensors;
1062 }
1063
1064 /**
1065  * 被 aten/src/ATen/native/Checkpoint.cpp 中的函数调用
1066  */
1067 // TODO: check that mutated value does not have alias. 检查变异值是否没有别名。
1068 /*对现有张量进行变异 (mutate) */
1069 void CheckpointTensorImpl::mutate(const std::string& name,
1070                                   const mutate_function_t& mutate,
1071                                   const Tensors& inputs,
1072                                   const std::vector<size_t>& mutate_idx) {
1073     // remat 是一个闭包，用于重新生成张量。
1074     auto remat = [=](const Tensors& t) -> Tensors {
1075         Tensors new_input_values = t;
1076         // 遍历 mutate_idx，对指定索引的张量进行克隆，并调用 mutate 函数对
其进行变异操作。
1077         for (size_t idx: mutate_idx) {
1078             new_input_values[idx] = t[idx].clone(); // 克隆要变异的张量
1079         }
1080         mutate(new_input_values); // 应用变异函数
1081         return new_input_values; // 返回新的张量列表
1082     };
1083
1084     // 尝试将输入张量检查点化
1085     Tensors checkpointed_inputs = try_checkpoint(inputs);
1086
1087     // 初始化用于存储输入张量值和日志记录参数的容器。
1088     strongs input_values;
1089     std::vector<std::string> args;
1090
1091     // 遍历检查点化后的输入张量，提取每个张量的值并存储在 input_values 中。如果启用了日
志记录功能，将每个张量的计数器名称添加到 args 中。
1092     for (const Tensor& t: checkpointed_inputs) {
1093         auto* cpti = dynamic_cast<CheckpointTensorImpl*>(t.unsafeGetTensorImpl());
1094         TORCH_CHECK(cpti);
1095         input_values.push_back(cpti->ref->value->value);
1096         if (use_log_) {
1097             args.push_back(cpti->counter_name());
1098         }

```

```

1099     }
1100
1101     // 调用 make_raw 函数进行重新生成，获取输出张量及其相关信息。
1102     auto ret = make_raw(remat, input_values);
1103
1104     // 遍历 mutate_idx，使用 cell_from_tensor 函数获取相应张量的引用，并更新其值为生成
    的输出张量值。
1105     const auto& modified = ret.outputs;
1106     for (size_t idx: mutate_idx) {
1107         cell_from_tensor(inputs[idx])->value = modified[idx];
1108     }
1109
1110     // 如果启用了日志记录，则记录变异操作
1111     if (use_log_) {
1112         DTRLogMutate(name, args, mutate_idx, from_time(ret.time));
1113     }
1114 }
1115
1116 /*释放 CheckpointTensorImpl 对象所持有的资源。这个函数在类的生命周期结束时调用，以确保
    所有持有的外部资源被正确释放。*/
1117 void CheckpointTensorImpl::release_resources() {
1118     // DTRLogRelease 记录了资源释放的日志，counter_name 是一个字符串，表示当前
    CheckpointTensorImpl 对象的唯一标识符。
1119     if (use_log_) {
1120         DTRLogRelease(counter_name());
1121     }
1122     // 将 ref 重置为 nullptr。
1123     ref.reset();
1124 }
1125
1126 /*CheckpointTensorImpl构造函数*/
1127 CheckpointTensorImpl::CheckpointTensorImpl(const Tensor& t) :
1128     // 这里使用初始化列表来调用另一个构造函数 CheckpointTensorImpl(const
    intrusive_ptr<External>& ext).
1129     // 调用 intrusive_ptr<External>::make 函数，用 t 创建一个 External 对象，并返回
    一个 intrusive_ptr<External>，该对象管理 External 的生命周期。
1130     CheckpointTensorImpl(intrusive_ptr<External>::make(t)) {
1131     // ref 是当前对象的一个成员变量，指向的是 intrusive_ptr<External>。
1132     // ref->value 是一个指向 External 的 intrusive_ptr，通过
    weak_intrusive_ptr<External> 创建一个弱引用。
1133     // 这个弱引用被添加到 CheckpointPool 对象 pool 的 exts 向量中。
1134     pool.exts.push_back(weak_intrusive_ptr<External>(ref->value));
1135 }
1136
1137 }

```

DELTA补充

```

1 // aten/src/ATen/CheckpointTensorImpl.cpp 代码
2 void CheckpointPool::auto_evict() {
3     auto device_stat = c10::cuda::CUDACachingAllocator::getDeviceStats(0);
4     long current_memory = device_stat.allocated_bytes[0].current; // 如果
    current_memory 是缓存的大小呢?
5     while (current_memory > memory_budget) {
6         evict();
7     }
8 }
9
10 void CheckpointPool::evict() {
11     TORCH_CHECK(aps.size() > 0);
12     bool shrunk = false;
13     int evict_idx = -1;
14     double evict_cost = INFINITY;
15     time_t current_time = std::chrono::system_clock::now();
16     auto remove_from_aps = [&](size_t i) {
17         aps[i] = aps[aps.size() - 1];
18         aps.pop_back();
19     };
20     // Find
21     for (size_t i = 0; i < aps.size(); i) {
22         auto cannot_evict = [&]() {
23             shrunk = true;
24             remove_from_aps(i);
25         };
26         auto ap_strong = aps[i].lock();
27         if (!ap_strong.defined()) {
28             cannot_evict();
29         }
30         else if (ap_strong->ecn) { // 已经被驱逐
31             cannot_evict();
32         }
33         else if (ap_strong->is_offloaded) { // 已经被卸载
34             cannot_evict();
35         }
36         else {
37             if (ap_strong->evictable()) {
38                 double cost = ap_strong->cost(current_time);
39                 if (cost < evict_cost) {
40                     evict_cost = cost;
41                     evict_idx = i;
42                 }
            }
        }
    }
}

```

```

43     }
44     i += 1;
45 }
46 }
47 std::cout << "aps.size(): " << aps.size() << std::endl;
48 std::cout << "evict_idx: " << evict_idx << std::endl;
49 if (evict_idx == -1) {
50     TORCH_CHECK(shrunk);
51 } else {
52     // Decision
53     auto release_aps = aps[evict_idx].lock();
54     double real_decision;
55     // if (release_aps->swapped) { // 情况1: 这个 tensor 曾经被交换过, 那就用记录
    // 的交换时间。但是, 这个真实的交换时间, 应该是换入+换出啊!!
56     //     double _compute_cost = static_cast<double>(release_aps->head_remat-
    >get_cpi().compute_cost);
57     //     double _swap_cost = release_aps->tensors[0].lock()->swap_cost;
58     //     real_decision = _swap_cost / _compute_cost;
59     // } else { // 情况2: 这个 tensor 还没被交换过, 不知道实际的交换时间, 就通过计算
    // 估计交换的时间。
60     //     real_decision = release_aps->head_remat-
    >get_cpi().decision(release_aps->memory);
61     // }
62
63     real_decision = release_aps->head_remat->get_cpi().decision(release_aps-
    >memory);
64     // Evict
65     auto evict_from_idx = [&](size_t idx) {
66         auto ap_strong = aps[idx].lock();
67         TORCH_CHECK(ap_strong.defined());
68         ap_strong->evict();
69         remove_from_aps(evict_idx);
70     };
71     // Offload
72     auto offload_from_idx = [&](size_t idx){
73         auto ap_strong = aps[idx].lock();
74         TORCH_CHECK(ap_strong.defined());
75         ap_strong->offload();
76         remove_from_aps(evict_idx);
77     };
78     if (real_decision > 1) { // recompute 成本低
79         evict_from_idx(evict_idx);
80         evict_count += 1;
81     } else { // swap 成本低
82         offload_from_idx(evict_idx);
83         offload_count += 1;
84     }

```

```

85     }
86 }
87
88 // 每次训练结束时调用该函数，打印并重新计数
89 void print_reset_count() {
90     std::cout << "evict_count: " << evict_count << std::endl;
91     std::cout << "offload_count: " << offload_count << std::endl;
92     evict_count = 0;
93     offload_count = 0;
94 }
95
96 void AliasPool::evict() {
97     TORCH_CHECK(!ecn); // 没有驱逐邻域，说明还没被驱逐。
98     ecn = head_remat->get_ecn();
99     auto ecns = neighbor_ecn();
100     for (const auto& necn : ecns) {
101         merge<CheckpointInfo>(merge_cpi, ecn, necn);
102     }
103     TORCH_CHECK(memory > 0);
104     TORCH_CHECK(lock_count == 0);
105     TORCH_CHECK(!is_evicted);
106     is_evicted = true;
107     onGPU = false;
108     for (const weak_intrusive_ptr<CheckpointTensorCell>& w : tensors) {
109         if (auto cell = w.lock()) {
110             cell->evict();
111         }
112     }
113 }
114
115 void AliasPool::offload() {
116     TORCH_CHECK(memory > 0);
117     TORCH_CHECK(lock_count == 0);
118     TORCH_CHECK(!is_offloaded);
119     onGPU = false;
120     is_offloaded = true;
121     swapped = true;
122     for (const weak_intrusive_ptr<CheckpointTensorCell>& w : tensors) {
123         if (auto cell = w.lock()) {
124             time_t pre = std::chrono::system_clock::now();
125             cell->cpuTensor = cell->t->to(DeviceType::CPU); // 在 tensor 层面进行的
126             // swap, 能不能在 storage 层面进行?
127             cell->offload();
128             time_t post = std::chrono::system_clock::now();
129             cell->swap_cost = (post - pre).count(); // 实际的交换时间
130         }
131     }
132 }

```

```

131 }
132
133 double AliasPool::cost(time_t current_time) {
134     if(!tensors.empty()) {
135         auto strongCell = tensors[0].lock();
136         if(strongCell && strongCell->t) {
137             free_mem = sides_free_mem(*(strongCell->t));
138         }
139     }
140     // 调用成本函数: 原 delta / 融合 delta + dtr
141     double ret = head_remat->get_cpi().cost(memory, (current_time -
last_used_time).count());
142
143     // 调用成本函数: 原 delta + free_mem
144     // ret = head_remat->get_cpi().cost(memory, (current_time -
last_used_time).count(), free_mem);
145
146     // 调用成本函数: 融合 delta + dtr 后加入更多参数
147     // ret = head_remat->get_cpi().cost(memory, (current_time -
last_used_time).count(), free_mem, recompute_times);
148
149     return ret;
150 }
151
152 // aten/src/ATen/CheckpointTensorImpl.h 代码
153 Tensor get() {
154     if (! t) {
155         if (evicted) {
156             TORCH_CHECK(remat);
157             remat->remat();
158         }
159         else if (offloaded) {
160             reload();
161         }
162         else {
163             TORCH_CHECK(false);
164         }
165     }
166     TORCH_CHECK(t);
167     TORCH_CHECK(! t->key_set().has(DispatchKey::Checkpoint));
168     pool->last_used_time = std::chrono::system_clock::now(); // last_used_time
只在这里更新, 但是怎么去掉 swap 的时间?
169     return *t;
170 }

```